PLANNING

REF:

00 - Chuẩn Research & Evidence

CHUẨN VIẾT BÁO CÁO, RESEARCH & EVIDENCE

1. Mục đích

1.1. Chuẩn trình bày làm việc của project

Tài liệu này xác định một chuẩn chung để các báo cáo theo tuần có thể tái sử dụng trực tiếp khi viết báo cáo cuối kỳ. Chuẩn này gồm định dạng văn bản, cấu trúc nghiên cứu, quy tắc evidence và cách liên kết artifact.

[D] Project working standard, áp dụng cho tài liệu hiện tại cho đến khi có template hoặc quy định chính thức của VNUK thay thế:

- Khổ giấy: A4, hướng dọc.
- Font nội dung: Times New Roman, 13 pt.
- Heading cấp 1: Times New Roman, 14 pt, bold.
- Heading cấp 2 và cấp 3: Times New Roman, 12 pt, bold.
- Giãn dòng nội dung: 1.5 lines.
- Căn lề đoạn văn: justified.
- Khoảng cách sau đoạn văn: 6 pt; không tạo khoảng cách bằng cách nhấn Enter nhiều lần.
- Lề trang làm việc: trái 3.0 cm; phải 2.0 cm; trên 2.0 cm; dưới 2.0 cm.

1.2. Cấu trúc báo cáo mỗi tuần

Cấu trúc tuần sử dụng IMRaD đã điều chỉnh cho đồ án phần mềm. IMRaD gốc gồm Introduction, Methods, Results và Discussion. Project bổ sung Plan/Research Question, Background/Literature, Implementation, Experimental Setup, Limitations, Deliverables và References để tạo evidence chain từ nghiên cứu đến sản phẩm.

- Plan / Research Question: tuần này đang trả lời câu hỏi kỹ thuật hoặc câu hỏi nghiên cứu nào; câu hỏi phải đủ cụ thể để có thể kiểm chứng.

- Background / Literature: tài liệu chính thức, paper, textbook hoặc lý thuyết cơ sở nào được sử dụng; nêu rõ nguồn nào support cho quyết định nào.

- Methods: cách tiếp cận, kiến trúc, thuật toán, giả thuyết và phương pháp kiểm chứng được lựa chọn.

- Implementation: module, API, logic, algorithm, state ownership, data flow và test thực tế được triển khai.

- Experimental Setup: mô tả input, board states, search depth, hardware, runtime/browser, software version, algorithm configuration, số lần chạy và các điều kiện khác. Nếu có benchmark thì phần này phải đủ cụ thể để có thể tái lập.

- Results / Evidence: test result, screenshot, commit/tag, benchmark, dataset, bảng số liệu hoặc graph thu được từ hệ thống.

- Discussion: giải thích kết quả, nguyên nhân, trade-off, failure case và các phần chưa thực hiện hoặc chưa đạt kỳ vọng.

- Limitations: phạm vi mà dữ liệu hoặc thí nghiệm hiện tại chưa chứng minh được; không được mở rộng kết luận vượt ra ngoài phạm vi này.

- Deliverables: các artifact bắt buộc phải tồn tại để tuần được xem là hoàn thành.

- References: toàn bộ source được sử dụng trong tuần, ưu tiên primary source, official documentation và academic source.

2. Evidence Rules

Mọi claim kỹ thuật trong báo cáo phải được phân loại theo một trong bốn mức sau:

- [A] Source Evidence: code, test, screenshot, log, benchmark, dataset, commit/tag hoặc artifact do chính project tạo ra.

- [B] Official / Academic Evidence: official documentation, standard, textbook, peer-reviewed paper hoặc nguồn học thuật có thể kiểm tra.

- [C] Inference: suy luận kỹ thuật dựa trên [A] và [B]. Inference không được trình bày như fact nếu chưa có evidence kiểm chứng trực tiếp.

- [D] Recommendation: quyết định thiết kế hoặc giải pháp được đề xuất cho project. Recommendation phải nêu rationale, trade-off và cách verify sau khi triển khai.

Quy tắc ưu tiên:

- [A] chứng minh hệ thống thực tế đang làm gì.

- [B] support cho lý thuyết, hành vi của tool/runtime hoặc claim học thuật.

- [C] dùng để nối evidence thành reasoning nhưng phải giữ nhãn inference.

- [D] dùng để ra quyết định tiếp theo; sau implementation phải quay lại thu [A] để xác nhận.

3. Những artifact bắt buộc phải được liên kết

Evidence không chỉ được mô tả bằng văn bản. Khi artifact tồn tại, báo cáo phải chứa link, path hoặc identifier đủ để truy ngược về nguồn.

- Link repository và commit/tag đại diện cho implementation evidence.

- Test report hoặc screenshot cho test pass/fail.

- Screenshot, GIF hoặc video ngắn cho hành vi UI/UX hoặc flow trực quan.

- File benchmark CSV/JSON khi có performance measurement.

- Raw dataset hoặc generated experiment dataset nếu kết quả được tổng hợp thành bảng/graph.

- Architecture diagram hoặc data-flow diagram nếu tuần đó thay đổi state ownership, module boundary, component boundary hoặc Rust/Wasm boundary.

- Citation/link cho paper, textbook hoặc official documentation đã được dùng để đưa ra quyết định.

- Với benchmark, phải liên kết cả raw data lẫn script/config dùng để chạy benchmark nếu chúng tồn tại.

- Với graph/table, phải xác định được graph/table được sinh ra từ file dữ liệu nào.

4. Quy tắc trình bày

- Danh sách dùng dấu "-", không dùng ký hiệu chấm tròn.

- Heading phải phân cấp rõ ràng theo dạng 1, 1.1, 1.2, 2, 2.1...

- Nội dung mô tả dùng văn xuôi học thuật; list chỉ dùng cho tiêu chí, deliverable, test case, artifact, điều kiện thí nghiệm hoặc các mục cần đối chiếu.

- Không viết "đã tối ưu", "nhanh hơn", "tốt hơn", "hiệu quả hơn" nếu chưa có số đo, test hoặc experimental evidence chứng minh.

- Không ghi kết luận vượt quá phạm vi dữ liệu đã thu.

- Thuật ngữ tiếng Anh được giữ nguyên ở lần đầu và giải thích nghĩa tiếng Việt nếu cần.

- Phân biệt rõ Observed Result, Interpretation và Recommendation.

- Không dùng screenshot thay cho raw data khi raw data có thể được lưu.

- Không dùng source code snippet thay cho mô tả method; code là evidence cho implementation, không phải toàn bộ phần Methods.**5. Cách chuyển weekly research log sang báo cáo cuối kỳ**

Final thesis không phải weekly diary. Mỗi tuần tạo ra evidence units; khi viết báo cáo cuối kỳ, các evidence units được regroup theo chủ đề và chương.

- Background / Literature của các tuần → Chương Cơ sở lý thuyết / Related Work.

- Methods / Design → Chương Phân tích và thiết kế hệ thống.

- Implementation → Chương Cài đặt / Hiện thực hệ thống.

- Experimental Setup + Results → Chương Thực nghiệm và kết quả.

- Discussion + Limitations → Chương Đánh giá, hạn chế và hướng phát triển.

- Deliverables và Evidence Matrix → phụ lục hoặc bảng truy vết nếu cần.

- Weekly References → bibliography/reference list của báo cáo cuối kỳ sau khi loại trùng và chuẩn hóa citation style.

6. Nguồn về cấu trúc nghiên cứu

6.1. Purdue OWL - Organization at the Document Level

IMRaD là một framework phổ biến cho research writing, nhưng cấu trúc phải được điều chỉnh theo discipline và genre. Project này dùng IMRaD làm backbone và bổ sung các phần cần thiết cho software engineering.

https://owl.purdue.edu/owl/graduate_writing/introduction_to_writing/documents/drafting-your-document/organization-at-the-document-level.pdf

6.2. VNUK - biểu mẫu và quy trình nghiên cứu

VNUK sử dụng các biểu mẫu theo vòng đời nghiên cứu như đề xuất/thuyết minh, triển khai và báo cáo tổng kết. Project áp dụng cùng logic evidence lifecycle nhưng điều chỉnh cho đồ án phần mềm.

- Nếu biểu mẫu VNUK đang dùng là tài liệu nội bộ hoặc yêu cầu đăng nhập, phải đính kèm chính file/form được giảng viên hoặc nhà trường cung cấp thay vì trích một URL không truy cập được.

- Chưa tìm thấy một nguồn công khai của VNUK xác nhận cụ thể font, lề và line spacing cho báo cáo này; vì vậy các con số ở mục 1.1 hiện được ghi rõ là [D] Project working standard, không phải quy định chính thức của VNUK.

7. Nguồn phương pháp nghiên cứu / Scientific Writing

7.1. Purdue OWL - Organization at the Document Level

Dùng để xác định cách tổ chức research document và cách điều chỉnh IMRaD theo loại tài liệu.

https://owl.purdue.edu/owl/graduate_writing/introduction_to_writing/documents/drafting-your-document/organization-at-the-document-level.pdf

7.2. A Guide for Writing in the Scientific Forum

Phần Methods phải đủ chi tiết để một người khác có thể hiểu và tái lập study. Trong project này, nguyên tắc đó được áp dụng bằng việc ghi fixture, software version, environment, algorithm configuration, benchmark protocol và raw data.

https://pmc.ncbi.nlm.nih.gov/articles/PMC3052777/

7.3. Quy tắc áp dụng cho project

- Weekly Research Question, Methods, Results và Discussion là evidence units chứ không phải chapter cuối kỳ.

- Final thesis phải tổng hợp theo research problem, architecture, algorithm, experiment và finding; không giữ nguyên thứ tự nhật ký tuần nếu thứ tự đó làm yếu lập luận.

- Claim lịch sử hoặc academic contribution ưu tiên primary paper.

- Claim về framework, runtime, language hoặc tool behavior ưu tiên official documentation.

- Claim về performance của chính project phải dựa trên benchmark của project, không dựa vào blog hoặc benchmark của hệ thống khác.

8. Retrospective Reconstruction vì sản phẩm đã tồn tại

Các tab theo tuần được dùng để tái cấu trúc tiến trình nghiên cứu và phát triển thành evidence chain. Chúng không được dùng để bịa lại lịch sử phát triển.

- Nếu một feature đã tồn tại trong sản phẩm, phần Methods mô tả cách hệ thống thực sự được xây; phần Results phải lấy từ code, test, benchmark, deployment hoặc lịch sử repo thực tế.

- Nếu không còn evidence feature được hoàn thành chính xác ở tuần hoặc ngày nào, ghi "implementation evidence hiện tại" thay vì khẳng định một mốc thời gian không có source.

- Không tạo commit/date giả để làm timeline trông đẹp hơn.

- Module name, file name, API name hoặc function name trong report chỉ được gọi là fact sau khi đối chiếu repo. Nếu chưa đối chiếu, ghi rõ là architecture mapping hoặc recommendation.

- Khi có repository evidence, tạo Evidence Matrix theo chuỗi: Report Claim → File/Function → Test → Commit/Tag → Benchmark/Result → Figure/Table.

- Nếu source hiện tại khác proposal ban đầu, báo cáo phải ghi lại thay đổi như một design evolution: initial plan → observed constraint → decision → implementation → evidence.

- Với việc chuyển TypeScript sang Rust/Wasm, không được viết "Rust nhanh hơn" như fact trước benchmark. Phải ghi rõ hypothesis, migration gate, equivalence tests và kết quả đo thực tế.

01 - Kiến trúc TypeScript → Rust/Wasm

KIẾN TRÚC TYPESCRIPT → RUST/WASM: DESIGN EVOLUTION VÀ QUYẾT ĐỊNH CÔNG NGHỆ

0. Evidence Status

[D] Architecture Decision + retrospective design reconstruction. Lịch sử được mô tả theo đúng baseline của đề cương: TypeScript ban đầu được chọn cho toàn bộ hệ thống, bao gồm game engine và các thuật toán. Rust/Wasm chỉ xuất hiện ở giai đoạn bắt đầu triển khai computational/search algorithms. Chỉ những phần đã đối chiếu repository, test log, commit hoặc benchmark mới được nâng thành [A] Source Evidence.

1. Plan / Research Questions

- RQ-ARCH-1: Khi đề cương ban đầu lựa chọn TypeScript cho toàn bộ hệ thống, điều kiện kỹ thuật nào đủ để biện minh cho việc thay đổi implementation language của computational core?

- RQ-ARCH-2: "Tối ưu" trong quyết định chuyển TypeScript → Rust/Wasm phải được định nghĩa và đo bằng metric nào?

- RQ-ARCH-3: Làm thế nào chuyển computational core sang Rust mà không làm thay đổi semantics của RuleSet, GameState và search algorithms?

- RQ-ARCH-4: Làm thế nào để web product và experiment runner dùng cùng một authoritative implementation thay vì hai code path khác nhau?

2. Background / Literature

2.1. Baseline của đề cương

Theo kế hoạch ban đầu, project sử dụng React/TypeScript cho toàn bộ website và TypeScript cho game logic/AI algorithms. Đây là lựa chọn hợp lý ở giai đoạn đầu vì một language/runtime giúp giảm integration complexity, tận dụng type checking và testing trong cùng ecosystem.

2.2. Workload thay đổi khi bắt đầu triển khai thuật toán

Từ giai đoạn Pattern Detection, Heuristic Evaluation và đặc biệt Minimax/Alpha-Beta, computational core phải lặp lại các thao tác generate move, apply/undo state, terminal detection, pattern scanning, evaluation, recursion, pruning, hashing và cache lookup trên số lượng node lớn. (trên nhiều trạng thái bàn cờ trong cây tìm kiếm). Note: số node thực tế phụ thuộc vào candidate generation, board fixture, search depth, pruning và time budget.
Đây là workload khác với React rendering và application orchestration.

Define:

Nodes Visited: số trạng thái search thực tế được AI duyệt trong một lượt, sau candidate filtering và Alpha-Beta pruning.

2.3. Rust và WebAssembly chỉ tạo rationale, không tự tạo kết luận hiệu năng

[B] Rust ownership cho phép compiler kiểm tra ownership/borrowing và cung cấp memory-safety guarantees mà không cần garbage collector. Rust cũng hướng tới zero-cost abstractions.

[B] WebAssembly được thiết kế để chạy hiệu quả, portable và có thể được JavaScript gọi trong web application. Các đặc tính này tạo cơ sở để đánh giá Rust/Wasm như một implementation candidate cho computational core, nhưng không chứng minh project thực tế nhanh hơn TypeScript.

2.4. Không dùng lập luận “JavaScript chậm”

[B] V8 là JavaScript/WebAssembly engine hiệu năng cao và có JIT/garbage collection được tối ưu. Vì vậy quyết định migration không được viết thành “TypeScript chậm nên chuyển Rust”. Câu hỏi khoa học phải là: trên workload và environment của project, cùng thuật toán/cấu hình thì runtime implementation nào đạt chi phí thực thi thấp hơn?

3. Methods / Decision Framework

3.1. Ba lớp “optimization” phải tách biệt

- Algorithmic optimization: giảm hoặc tái sử dụng lượng computational work. Ví dụ Alpha-Beta Pruning, Candidate Move Generation, Move Ordering, Transposition Table.

- Runtime / implementation optimization: giảm execution cost của cùng workload. Metric gồm search time, cost per node, nodes/second, allocation/memory behavior và achieved depth trong cùng time budget.

- System optimization: tách computational core khỏi UI, tránh duplicate algorithms và cho phép web product + experiment runner dùng cùng implementation.

3.2. Alternatives được xem xét

- Alternative A: tiếp tục dùng TypeScript cho toàn bộ hệ thống. Ưu điểm là integration đơn giản và không có Wasm boundary; nhược điểm cần được đánh giá bằng workload search thực tế, không suy đoán.

- Alternative B: giữ React/TypeScript cho web layer và chuyển authoritative computational/game core sang Rust, expose coarse-grained API qua WebAssembly. Đây là architecture được chọn để nghiên cứu và triển khai.

- Alternative C: viết lại cả frontend bằng Rust/WebAssembly. Không được chọn vì không trực tiếp phục vụ research question và làm tăng migration scope không cần thiết.

3.3. Chronology của quyết định

- Proposal / Week 1: TypeScript được planned cho toàn bộ system, bao gồm algorithms.

- Week 2: xây dựng TypeScript Game Engine và correctness baseline.

- Week 3: React UI/PvP tích hợp với TypeScript Game Engine.

- Week 4: bắt đầu computational algorithms; đánh giá lại runtime/core architecture; tạo migration gate TypeScript → Rust/Wasm; triển khai Pattern/Heuristic/baseline bots trong Rust.

- Week 5+: Minimax, Alpha-Beta và các search optimizations được triển khai trong Rust core.

Chronology này là retrospective evidence organization, không được dùng để bịa commit date. Nếu repository không chứng minh đúng mốc tuần, report ghi "implementation evidence hiện tại".

4. Implementation Architecture

4.1. Ownership sau migration

- React/TypeScript sở hữu UI, input, routing, screen/application state, visualization và adapter.

- Rust sở hữu authoritative Core Game Engine cần cho AI computation: Board/SearchState, RuleConfig tương thích với game rules, legal-move semantics, apply/undo, terminal detection, pattern/evaluation, search algorithms, optimizations và metrics.

- WebAssembly là bridge cho web product.

- Experiment runner gọi cùng Rust core ở chế độ native/headless hoặc shared crate.

- React không chứa duplicated win-rule/search logic.

4.2. Target workspace

workspace/

- crates/caro-core/

  - model.rs

  - rules.rs

  - state.rs

  - pattern.rs

  - evaluator.rs

  - movegen.rs

  - metrics.rs

  - bot/random.rs

  - bot/rule_based.rs

  - search/minimax.rs

  - search/alpha_beta.rs
  - search/ordering.rs
  - search/iterative.rs
  - search/transposition.rs
  - zobrist.rs
- crates/caro-wasm/
  - wasm-bindgen adapter; không chứa algorithm implementation riêng
- crates/caro-experiments/
  - fixture runner, bot-vs-bot runner, CSV/JSON exporter
- web/
  - React/TypeScript UI + EnginePort/RustWasmAdapter

4.3. Boundary contract

JS↔Wasm calls phải coarse-grained:

- load_position(board, player, ruleConfig)
- make_move(row, col)
- choose_move(searchConfig)
- run_fixture(experimentConfig)

Recursive search, evaluation, apply/undo, hashing và transposition lookup nằm trọn trong Rust core. Không thiết kế JS gọi Rust cho từng cell, pattern hoặc search node.

5. Correctness Migration Gate

Trước performance benchmark, migration phải vượt qua correctness gate:

- cùng board encoding và Rule Config;
- cùng fixed fixtures;
- cùng legal-move result;

- cùng terminal/win/draw result;

- cùng apply/undo invariant;

- cùng evaluator weights và pattern taxonomy khi đối chiếu tương đương;

- cùng search depth/candidate policy/move order/tie-break khi so algorithm equivalence;

- state trước và sau search không bị thay đổi;

- Minimax và Alpha-Beta trả cùng minimax root value trên cùng configuration;

- serialization/deserialization qua Wasm boundary bảo toàn state.

Nếu TypeScript implementation tương đương không còn tồn tại hoặc không thể tái tạo từ repository history, không được tạo baseline giả để tuyên bố Rust nhanh hơn.

6. Experimental Setup cho TypeScript → Rust Comparison

6.1. Controlled variables

- Hardware/CPU/RAM/OS.

- Browser/Node/native runtime version.

- Build mode: production/release; Rust dùng release build.

- Board fixtures và side-to-move.

- RuleConfig.

- Algorithm và evaluator.

- Search depth hoặc fixed time budget.

- Candidate policy, move ordering và tie-break.

- Warm-up protocol và số lần lặp.

- Wasm initialization time tách khỏi search time nếu mục tiêu là đo search runtime.

6.2. Metrics

CostPerNode = SearchTime / NodesVisited

Throughput = NodesVisited / SearchTime

Speedup = T_TypeScript / T_Rust

DepthGain(B) = Depth_Rust(B) - Depth_TypeScript(B), với cùng time budget B.

Performance claim chỉ hợp lệ khi correctness và workload equivalence đã được chứng minh.

7. Results / Evidence

[A] Chỉ điền số liệu sau khi lấy từ repository/test/benchmark thật.

Bảng evidence tối thiểu:

- Report claim → file/function → test → commit/tag → benchmark/result → figure/table.

- Raw benchmark phải lưu CSV/JSON + environment metadata.

- Screenshot chỉ là evidence cho UI/integration; không thay thế raw benchmark.

- Graph phải link tới raw data và script/config sinh graph.

8. Discussion

Giá trị của migration không chỉ nằm ở speed. Architecture sau migration tạo một computational core độc lập React, có thể tái sử dụng cho product và experiment, đồng thời cho phép metrics được thu trực tiếp từ algorithm implementation. Tuy nhiên, lợi ích runtime là empirical claim và phải được benchmark.

Algorithmic optimization và runtime optimization không được trộn nguyên nhân. Nếu Alpha-Beta giảm nodesVisited thì đó là tác động thuật toán. Nếu Rust xử lý cùng nodesVisited trong ít thời gian hơn thì mới là runtime implementation effect.

9. Limitations

- Wasm boundary có conversion/copy overhead tùy data shape.

- Workload nhỏ có thể không hưởng lợi từ migration.

- V8/JIT performance phụ thuộc engine/environment; kết quả một máy không đại diện mọi platform.

- Rule profile Caro/Gomoku phải được khóa chính xác.

- Reference code chưa được đối chiếu production repository không được gắn [A].

- Rust/Wasm migration làm tăng build/debug/toolchain complexity.

10. Deliverables / Definition of Done

- Architecture decision record nêu đúng baseline TypeScript ban đầu và migration tại Week 4.

- TypeScript correctness fixtures từ Week 2/3 được giữ làm migration oracle nếu tồn tại.

- Rust core build độc lập React.

- Wasm adapter end-to-end.

- Web product và experiment runner gọi cùng Rust core.

- Equivalence tests pass trước benchmark.

- Raw benchmark CSV/JSON + environment metadata tồn tại trước mọi performance conclusion.

- Evidence matrix được cập nhật bằng repository thật.

11. References

- [TS-1] TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/

- [VITEST-1] Vitest Guide: https://vitest.dev/guide/

- [RUST-1] The Rust Programming Language, Introduction: https://doc.rust-lang.org/book/ch00-00-introduction.html

- [RUST-2] Understanding Ownership: https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

- [WASM-1] MDN, WebAssembly Concepts: https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Concepts

- [WASM-2] wasm-bindgen Guide: https://wasm-bindgen.github.io/wasm-bindgen/

- [WASM-3] wasm-bindgen numeric slice transfer notes: https://wasm-bindgen.github.io/wasm-bindgen/reference/types/boxed-number-slices.html

- [V8-1] V8 Documentation: https://v8.dev/docs

- [AI-1] UC Berkeley CS188, Minimax/Alpha-Beta: https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

- [H-1] A. Zobrist (1970), A New Hashing Method With Application for Game Playing: https://research.cs.wisc.edu/techreports/1970/TR88.pdf

Đúng

Cụ thể:

- `[A]` Board 15×15 có không gian trạng thái và game tree tăng rất nhanh về mặt lý thuyết.

- `[B]` Rust có các đặc tính phù hợp cho computational core: kiểm tra ownership, không GC, có thể build sang Wasm.

- `[C]` Từ hai ý trên chỉ suy ra: Rust/Wasm là **implementation candidate hợp lý** cho phần search/evaluation.

- Không thể suy ra ngay: "vì thế Rust nhanh hơn TS trong game này." Điều đó phải benchmark.

Ý của bạn là đúng hướng, nhưng nên viết theo kiểu **nghiên cứu/đánh giá**, không phải "chọn Rust trước vì đã biết nó nhanh":

> Ưu tiên hiện thực computational core bằng Rust và biên dịch sang WebAssembly để đánh giá tính phù hợp của hướng tiếp cận này. Sau đó, xây dựng hoặc giữ một implementation TypeScript tương đương làm baseline. Hai implementation được chạy trên cùng thuật toán, board fixture, search depth, candidate-generation policy, Alpha-Beta policy và time budget để đo các chỉ số `SearchTime`, `NodesVisited`, `CostPerNode`, `Throughput` và achieved depth.

>

> Mục tiêu không phải giả định Rust nhanh hơn TypeScript, mà kiểm chứng liệu Rust/Wasm có giảm chi phí thực thi cho cùng computational workload hay không.

Có một điểm quan trọng: **không nên implement Rust với thuật toán tốt hơn, rồi implement TS đơn giản hơn để so sánh**. Khi đó kết quả chỉ chứng minh khác biệt thuật toán, không chứng minh khác biệt runtime/language.

So sánh công bằng nên là:

```text
Cùng board fixture
```

+ cùng side-to-move

+ cùng candidate moves

+ cùng heuristic/evaluation weights

+ cùng move ordering

+ cùng Alpha-Beta / transposition-table policy

+ cùng max depth hoặc time budget

= chỉ khác implementation runtime: TypeScript vs Rust/Wasm

```

Nếu TS implementation đã có sẵn trong project, cách trình bày còn tự nhiên hơn:

> TypeScript implementation hiện có được xem là baseline. Rust/Wasm core được xây dựng với cùng semantic contract và được benchmark đối chiếu với baseline để xác định runtime implementation nào đạt throughput tốt hơn dưới cùng workload.

Còn nếu chưa có implementation nào cả, thì đừng nói "implement Rust trước, sau đó làm lại TS" như thể TS chỉ để đối chứng sau. Nên ghi:

> Xác định trước một thuật toán và semantic contract chung; sau đó hiện thực hai phiên bản Rust/Wasm và TypeScript tương đương để benchmark.

Thực tế bạn vẫn có thể code Rust trước vì mục tiêu chính là thử hướng Rust/Wasm. Nhưng trong thesis/report, phần kết luận cần giữ trung lập: **Rust là hypothesis/candidate; benchmark mới là bằng chứng.**
```

02 - Mathematical Foundations

MATHEMATICAL FOUNDATIONS CHO ĐỒ ÁN CỜ CARO AI

Mục tiêu của tab này là khóa toàn bộ ký hiệu, công thức và metric toán học cần dùng xuyên suốt báo cáo. Không cần nhồi công thức vào mọi chương; mỗi công thức chỉ được dùng khi nó giúp định nghĩa algorithm, experiment hoặc interpretation.

1. Ký hiệu chung

- N: kích thước một chiều của bàn cờ. Với board 15×15, N = 15.

- $C = N^2$: tổng số ô. Với N = 15, C = 225.

- s: một game state.

- a: một action/move.

- A(s): tập legal actions tại state s.

- p: player hiện tại.

- b: branching factor, số child trung bình được mở rộng tại một node.

- m: search depth tính theo ply.

- n: số mẫu benchmark hoặc số ván thực nghiệm.

- T: thời gian chạy/search time.

- V(s): Minimax value của state s.

- E(s,p): heuristic evaluation của state s theo góc nhìn player p.

- α: lower bound tốt nhất MAX đã bảo đảm trên path hiện tại.

- β: upper bound tốt nhất MIN đã bảo đảm trên path hiện tại.

2. Board indexing

Nếu board 2D được flatten thành mảng một chiều:

index = row × N + col

row = floor(index / N)

col = index mod N

Với N = 15:

index = 15 × row + col

Công thức này cần hiểu vì Rust core có thể dùng [i8; 225] hoặc buffer tuyến tính thay cho nested arrays.

3. Upper bound thô của state configurations

Nếu mỗi cell chỉ có ba symbol {empty, player1, player2}, số cấu hình biểu diễn thô là:

S_upper = 3^(N²)

Với N = 15:

S_upper = 3^225

Đây KHÔNG phải số legal game states chính xác vì nó chứa nhiều cấu hình không thể đạt được do turn counts, terminal conditions và luật game. Nó chỉ minh họa độ lớn của không gian biểu diễn.

4. Branching factor và game-tree growth

Nếu mỗi node có trung bình b child và search đến depth m, số node của full uniform tree xấp xỉ:

Nodes(m) = Σ(d=0..m) b^d

Nếu b ≠ 1:

Nodes(m) = (b^(m+1) - 1) / (b - 1)

Khi b > 1 và m tăng, dominant term là b^m nên Minimax baseline có time complexity:

O(b^m)

Ví dụ chỉ để trực giác, không dùng làm benchmark thật:

- b = 20, m = 3 → khoảng 1 + 20 + 400 + 8000 = 8421 nodes.

- b = 20, m = 5 → hơn 3.3 triệu nodes.

Nguồn đọc: UC Berkeley CS188 Minimax / Alpha-Beta

https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

5. Minimax recurrence

Với zero-sum two-player game:

V(s) = terminalUtility(s), nếu s là terminal.

Nếu s là MAX node:

$V(s) = \max[a \in A(s)] V(Result(s,a))$

Nếu s là MIN node:

$V(s) = \min[a \in A(s)] V(Result(s,a))$

Khi depth limit bằng 0 mà state chưa terminal:

$V(s) \approx E(s,p)$

Ký hiệu Result(s,a) là state nhận được sau khi áp dụng action a vào state s.

6. Heuristic evaluation

Một dạng tổng quát:

$$E(s,p) = Attack(s,p) - \lambda \times Defense(s,opponent)$$

Trong đó λ là hệ số thể hiện mức ưu tiên phòng thủ. Nếu $\lambda = 1$ thì hai phía cân bằng theo scale đang dùng.

Pattern-based score:

$$Attack(s,p) = \Sigma_i w_i \times count_i(s,p)$$

$$Defense(s,q) = \Sigma_i v_i \times count_i(s,q)$$

- i: pattern class, ví dụ Open Four, Closed Four, Open Three.
- w_i: attack weight của pattern i.
- v_i: defense weight của pattern i.
- $count_i$: số pattern loại i trong state.

Weights là parameter của model heuristic, không phải định luật toán học. Báo cáo phải ghi version của weight table và cách chọn/tune.

7. Terminal-score dominance

Để Minimax không chọn heuristic đẹp hơn một forced win, terminal utility cần nằm ngoài khoảng heuristic non-terminal.

Nếu $|E(s,p)| \leq H_max$ cho mọi state non-terminal thì chọn:

$WIN_SCORE > H_max$

$LOSS_SCORE = -WIN_SCORE$

Nếu muốn ưu tiên thắng nhanh hơn:

$TerminalWinScore = WIN_SCORE - plyFromRoot$

$TerminalLossScore = -WIN_SCORE + plyFromRoot$

Cách này phải được áp dụng nhất quán trong Minimax và Alpha-Beta.

8. Alpha-Beta pruning

Alpha và beta ban đầu:

$\alpha = -\infty$

$\beta = +\infty$

MAX update:

$\alpha = \max(\alpha, \text{childValue})$

MIN update:

$\beta = \min(\beta, \text{childValue})$

Cutoff condition:

$\alpha \geq \beta$

Khi điều kiện này đúng, các child còn lại của node hiện tại không thể thay đổi Minimax decision theo bound hiện có nên có thể prune.

Worst case nếu move ordering xấu vẫn cùng bậc với Minimax:

$O(b^m)$

Trong ideal/perfect ordering, lý thuyết thường mô tả Alpha-Beta có effective complexity gần:

$O(b^{(m/2)})$

Không dùng $O(b^{(m/2)})$ như lời hứa runtime thực tế. Nó là best-case theoretical behavior phụ thuộc ordering.

Nguồn đọc: UC Berkeley CS188 Alpha-Beta

https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

9. Node reduction

Để so optimization với baseline:

NodeReduction = 1 - N_opt / N_base

NodeReductionPercent = 100 × (1 - N_opt / N_base)

- N_base: nodesVisited của baseline.
- N_opt: nodesVisited sau optimization.

Ví dụ N_base = 100000, N_opt = 25000:

NodeReduction = 0.75 = 75%

10. Speedup

Speedup = T_baseline / T_optimized

- Speedup > 1: optimized implementation nhanh hơn theo metric timing đó.
- Speedup = 1: không khác.
- Speedup < 1: optimized implementation chậm hơn.

Không dùng phần trăm “nhanh hơn” nếu chưa định nghĩa rõ cách tính. Speedup ratio thường ít mơ hồ hơn.

11. Candidate Move Generation reduction

Nếu L = số legal moves và C = số candidate moves được search:

CandidateRatio = C / L

CandidateReduction = 1 - C / L

Nhưng giảm C không tự động chứng minh search tốt hơn: phải kiểm tra tactical recall để tránh loại mất nước đúng.

12. Tactical recall cho candidate generator

Nếu có tập fixture với expected critical moves:

Recall = criticalMovesRetained / totalCriticalMoves

Mục tiêu nghiên cứu không chỉ là CandidateReduction cao mà còn giữ Recall gần 1 trên tactical corpus.

13. Prune rate

Phải khóa definition trước khi đo. Một lựa chọn:

PruneRate = N_pruned / (N_visited + N_pruned)

Nhưng “node pruned” đôi khi không thể đếm trực tiếp nếu subtree chưa được materialize. Một định nghĩa khác có thể là cutoff count hoặc skipped-child count. Vì vậy báo cáo phải ghi metric definition chính xác và không trộn các definition giữa experiment.

14. Mean và median cho benchmark

Arithmetic mean:

$$\text{mean}(T) = (1/n) \times \Sigma_i T_i$$

Median:

- sắp xếp T_i tăng dần.
- nếu n lẻ: lấy phần tử giữa.
- nếu n chẵn: trung bình hai phần tử giữa.

Timing trong browser có thể có outlier do scheduler/JIT/background activity; median thường hữu ích để mô tả typical runtime, nhưng nên lưu raw samples và báo thêm spread.

Nguồn NIST - Measures of Location:

https://www.itl.nist.gov/div898/handbook/eda/section3/eda351.htm

15. Variance và standard deviation

Sample variance:

$$s^2 = [\Sigma_i (T_i - \bar{T})^2] / (n - 1)$$

Sample standard deviation:

$s = \sqrt{s^2}$

Standard deviation cho biết mức phân tán quanh mean. Với benchmark skew/outlier mạnh, kết hợp median + percentile thường dễ diễn giải hơn chỉ mean ± SD.

16. Percentile

p50 ≈ median.

p95 = giá trị mà khoảng 95% samples không vượt quá.

Trong game search, p95SearchTime hữu ích vì một bot có median tốt nhưng thỉnh thoảng đứng lâu vẫn tạo UX tệ.

17. Win rate

Nếu bot thắng W ván trên tổng n ván:

$p_hat = W / n$

Nếu tính draw riêng:

WinRate = W/n

DrawRate = D/n

LossRate = L/n

W + D + L = n

Không nên kết luận “bot A mạnh hơn” chỉ từ vài ván. Khi số ván đủ, cần confidence interval cho proportion.

18. Wilson confidence interval cho tỷ lệ thắng

Với p_hat = W/n, z là critical value (z ≈ 1.96 cho 95% CI):

center = [p_hat + $z^2/(2n)$] / [1 + z^2/n]

half = [z / (1 + z^2/n)] × sqrt[p_hat(1-p_hat)/n + $z^2/(4n^2)$]

CI = [center - half, center + half]

Nguồn NIST về confidence interval cho binomial proportions:

https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm

19. Zobrist hashing

Cho mỗi cell i và piece p, sinh một random bitstring Z[i,p]. Hash của position là XOR các entry đang có quân:

H(s) = XOR over occupied (i,p) of Z[i,p]

Khi thêm/bỏ quân tại cell i:

H_new = H_old XOR Z[i,p]

Do XOR có tính self-inverse:

x XOR y XOR y = x

nên apply/undo có thể update hash incrementally mà không hash lại toàn board.

Nguồn gốc:

Albert L. Zobrist, A New Hashing Method With Application for Game Playing, 1970.

https://research.cs.wisc.edu/techreports/1970/TR88.pdf

20. Transposition-table hit rate

Nếu lookups = tổng số lần search hỏi TT và hits = số lần tìm thấy usable entry:

TTHitRate = hits / lookups

Metric này chỉ có nghĩa khi định nghĩa "usable hit" rõ: depth đủ? bound type phù hợp? generation còn valid?

21. Iterative Deepening

Search lần lượt depth 1,2,...,d cho tới time budget.

AchievedDepth = max depth hoàn tất trước deadline.

Nếu t_d là thời gian vòng depth d:

TotalTime(d) = Σ(k=1..d) t_k

Mặc dù lặp lại node ở depth nhỏ, ordering/TT từ vòng trước có thể giúp vòng sâu hơn. Lợi ích phải đo thay vì chỉ giả định.

22. Time budget

Nếu budget là B milliseconds và t_start là thời điểm bắt đầu:

remaining = B - (now - t_start)

Search phải có cancellation/deadline checks đủ thưa để overhead không quá lớn nhưng đủ dày để response không vượt budget nghiêm trọng. Tần suất check là engineering parameter cần benchmark.

23. Experimental fairness

Khi so A và B, chỉ thay một independent variable chính nếu muốn suy ra causal explanation.

Ví dụ Alpha-Beta vs Minimax:

Control variables = board, depth, evaluator, move generator, move ordering, tie-break, hardware/runtime.

Independent variable = pruning enabled/disabled.

Dependent variables = nodesVisited, searchTime, nodesPruned.

Nếu đổi cả move ordering cùng lúc thì không thể quy toàn bộ chênh lệch cho Alpha-Beta.

24. Bộ công thức tối thiểu phải thuộc trước khi bảo vệ

- index = row×N + col.
- 3^(N²) chỉ là upper bound thô của configurations.
- Nodes ≈ Σ b^d và Minimax O(b^m).
- Minimax MAX = max(child), MIN = min(child).
- E = Attack - Defense và weighted pattern sum.
- Alpha-Beta cutoff khi $\alpha \geq \beta$.
- NodeReduction = 1 - N_opt/N_base.
- Speedup = T_base/T_opt.
- mean, median, standard deviation.
- WinRate = W/n và ý nghĩa confidence interval.
- Zobrist = XOR random keys.

25. Reading path toán học

[MATH-1] Berkeley CS188 Game Trees / Minimax / Alpha-Beta

https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

[MATH-2] NIST Measures of Location

https://www.itl.nist.gov/div898/handbook/eda/section3/eda351.htm

[MATH-3] NIST Confidence Intervals for Proportions

https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm

[MATH-4] Zobrist 1970 Technical Report

https://research.cs.wisc.edu/techreports/1970/TR88.pdf

[MATH-5] Allis thesis - Searching for Solutions in Games and Artificial Intelligence

https://doi.org/10.26481/dis.19940923la

26. Runtime migration metrics TypeScript → Rust/Wasm

Phần này chỉ dùng khi có TypeScript implementation tương đương đủ evidence để làm baseline. Nếu historical baseline không tồn tại, không được dựng lại một implementation mới rồi gọi đó là "kết quả trước migration".

26.1. Cost per node

Với T là search time và N là nodesVisited:

$$CostPerNode = T / N$$

Metric này hữu ích khi hai implementation thực hiện cùng search workload. Nếu N khác nhau vì candidate policy, ordering hoặc pruning khác nhau thì CostPerNode không còn cô lập runtime effect hoàn toàn.

26.2. Throughput

$$Throughput = NodesVisited / SearchTime$$

Có thể biểu diễn nodes/second. Throughput cao hơn chỉ có ý nghĩa khi node semantics và workload tương đương.

26.3. Runtime speedup

$$Speedup_{TS \rightarrow Rust} = T_{TS} / T_{Rust}$$

- Speedup > 1: Rust implementation hoàn thành workload đó trong ít thời gian hơn.

- Speedup = 1: không có khác biệt theo timing metric đó.

- Speedup < 1: Rust implementation chậm hơn trên workload/environment đó.

Không suy rộng kết quả một fixture/machine thành kết luận universal về hai ngôn ngữ.

26.4. Depth gain dưới cùng time budget

Với fixed budget B:

DepthGain(B) = Depth_Rust(B) - Depth_TS(B)

Metric này sát gameplay hơn raw microbenchmark nếu bot dùng iterative deepening/time budget. Cần bảo đảm cùng evaluator, candidate policy, ordering và correctness contract.

26.5. Wasm boundary overhead

Nếu T_end_to_end là latency từ JS call tới result và T_core là thời gian search được đo trong Rust core:

BoundaryAndIntegrationCost = T_end_to_end - T_core

BoundaryRatio = BoundaryAndIntegrationCost / T_end_to_end

Không gộp Wasm initialization/first-load cost vào search-core timing nếu research question chỉ đo thuật toán. Nếu research question là user-perceived latency thì phải dùng end-to-end metric và ghi rõ.

26.6. Fair-comparison contract

Để quy chênh lệch timing cho runtime implementation, phải giữ cố định:

- board fixture;

- RuleSet;

- algorithm;

- search depth hoặc time/node budget;
- evaluator và weights;
- candidate generator;
- move ordering;
- tie-break;
- build mode;
- hardware/OS/runtime versions;
- warm-up và repeated-sample protocol.

Nếu đổi nhiều hơn một independent variable thì conclusion phải mô tả combined-system effect thay vì claim riêng Rust effect.

01W - Tuần 1 - Research Foundation

TUẦN 1 - RESEARCH FOUNDATION, RULES, STATE SPACE VÀ SCOPE

0. Artifact phải đính kèm

- Đề cương đồ án chính thức.

- Rule Specification phiên bản 1.

- Danh sách Research Questions của toàn đồ án.

- Reading notes từ các paper/docs chính.

- Bảng terminology: Gomoku, Caro, state, state space, game tree, branching factor, ply.

- Scope / Out-of-scope được khóa.

1. Research objective

Tuần 1 không chỉ "đọc lý thuyết". Output bắt buộc là một Research Contract đủ chính xác để mọi code, experiment và conclusion ở tuần sau dùng cùng luật, cùng thuật ngữ và cùng metric.

2. Research Questions

- Rule profile nào được project triển khai và điều kiện thắng được định nghĩa chính xác ra sao?

- State của game cần chứa dữ liệu gì để vừa phục vụ gameplay vừa phục vụ search?

- Vì sao full game-tree search không khả thi trên board 15×15?

- Những thuật toán nào nằm trong scope chính thức và metric nào sẽ dùng để so sánh?

- Công trình trước đây về Gomoku cho thấy vai trò của search và domain knowledge như thế nào?

3. Background / Literature

Gomoku là game hai người trên grid, nhưng có nhiều biến thể luật. Vì vậy báo cáo phải phân biệt rõ luật quốc tế được trích từ nguồn và rule profile riêng của project.

Không gọi “Caro Việt Nam chặn hai đầu” là chuẩn quốc tế nếu không có nguồn chính thức xác nhận.

Nguồn luật cần đọc:

[R1] Renju International Federation - International Rules of Gomoku

https://gomoku.renju.net/gomokurules/

Công trình của Allis et al. về Gomoku là nguồn nền tảng cho related work. Giá trị chính đối với project là chứng minh search space lớn và domain-specific threat knowledge có thể hỗ trợ search. Project không tự động triển khai Threat-Space Search hoặc Proof-Number Search nếu chúng không nằm trong scope đề cương.

[R2] Allis, van den Herik & Huntjens (1993), Go-Moku Solved by New Search Techniques

https://cdn.aaai.org/Symposia/Fall/1993/FS-93-02/FS93-02-001.pdf

[R3] Allis (1994), Searching for Solutions in Games and Artificial Intelligence

https://doi.org/10.26481/dis.19940923la

4. Formal problem definition

Input:

- RuleProfile.

- Board state.

- Current player.

Process:

- Validate move.

- Apply state transition.

- Detect terminal state.

- Khi AI được thêm: generate/evaluate/search moves.

Output:

- Updated state hoặc SearchResult.

Constraints:

- Board 15×15 nếu scope cuối cùng giữ cấu hình này.

- Hai người chơi, deterministic, zero-sum ở lớp search abstraction.

- Branching factor cao nên full exhaustive search không khả thi ở depth lớn.

5. Board/state representation decision

Logical board là ma trận N×N. Cell encoding phải khóa một lần và được dùng xuyên TS/Rust/dataset.

Đề xuất:

- EMPTY = 0.

- PLAYER_A = 1.

- PLAYER_B = -1.

- row, col chạy từ 0 đến N-1.

- lastMove được lưu để win detection không phải scan toàn board.

Nếu implementation thực tế đang dùng 1/2 thay vì ±1, báo cáo phải đổi theo code thật. Không sửa code chỉ để khớp tài liệu.

6. State space và game tree

Upper bound biểu diễn thô với 3 trạng thái/cell là $3^{(N^2)}$, nhưng đây không phải số legal states chính xác. Game Tree có node = state, edge = legal move. Branching factor b và depth m dẫn đến Minimax baseline $O(b^m)$, là lý do project cần Candidate Move Generation, Alpha-Beta, Move Ordering và các tối ưu sau.

Công thức chi tiết nằm trong tab “02 - Mathematical Foundations”.

7. Scope chính thức

Algorithms:

- Random Move.

- Rule-Based / Heuristic.

- Minimax.

- Alpha-Beta Pruning.

- Pattern Detection.

- Candidate Move Generation.
- Move Ordering.
- Iterative Deepening.
- Transposition Table.
- Zobrist Hashing.

Metrics:

- searchTimeMs.
- nodesVisited.
- nodesPruned hoặc cutoff metric được định nghĩa rõ.
- achievedDepth.
- candidate count.
- evaluation score.
- win/draw/loss trong Bot-vs-Bot.

8. Evidence model

- [A] Source evidence: code, test, benchmark, screenshot, CSV/JSON.
- [B] Academic/official evidence: paper, textbook, official docs.
- [C] Inference: suy luận từ A+B.
- [D] Recommendation: quyết định thiết kế cho project.

Mọi kết luận trong final report phải biết mình thuộc nhóm nào.

9. Methodology output của Tuần 1

- Viết Rule Specification có examples và edge cases.
- Viết Research Questions.
- Khóa glossary và metric definitions.
- Lập reading list.

- Ghi nhận baseline công nghệ của đề cương: React/TypeScript được planned cho toàn bộ hệ thống, bao gồm Core Game Engine và AI algorithms. Tuần 1 chưa có quyết định Rust; migration chỉ được đánh giá ở Week 4 khi computational workload bắt đầu xuất hiện.

- Không benchmark ngôn ngữ ở Tuần 1; chỉ định nghĩa protocol sẽ dùng sau.

10. Definition of Done

- Rule profile rõ exact-five/overline/blocking behavior.

- Board size, encoding và coordinates đã khóa.

- Scope algorithms không mâu thuẫn đề cương.

- Metrics có definition.

- Research Questions có thể trả lời bằng experiment về sau.

- Primary sources đã lưu link/citation.

- Không còn claim học thuật không có source.

11. Reading checklist

- R1: đọc toàn bộ basic rules và win condition.

- R2: đọc Abstract, rules, threat definitions, search overview, conclusion.

- R3: đọc phần liên quan game solving/Gomoku khi viết literature review.

- Berkeley CS188 để chuẩn bị search complexity:

https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

12. Output chuyển thẳng sang luận án

- Problem statement.

- Game/rule definition.

- State-space discussion.

- Related work về Gomoku search.

- Research questions.

- Scope và limitations ban đầu.

02W - Tuần 2 - TypeScript Game Engine

TUẦN 2 - TYPESCRIPT CORE GAME ENGINE, RULES, STATE VÀ CORRECTNESS BASELINE

0. Artifact phải đính kèm

- Đề cương và technology baseline từ Tuần 1: TypeScript được planned cho toàn bộ hệ thống.

- Rule Specification phiên bản đang sử dụng.

- Repository commit/tag của TypeScript Game Engine nếu repository history còn lưu.

- Source files hoặc architecture mapping cho Board, GameState, RuleSet và MoveResult.

- Vitest/unit-test report.

- Fixture corpus cho legal move, win/draw và edge cases.

- State ownership/data-flow diagram.

1. Plan / Research Questions

Tuần 2 tập trung vào correctness của domain model bằng TypeScript, đúng với kế hoạch công nghệ ban đầu của đề cương. Tuần này chưa có quyết định chuyển Rust.

- RQ-W2-1: Data model nào biểu diễn game state deterministic và đủ cho gameplay/search về sau?

- RQ-W2-2: Component/module nào phải sở hữu state transition để UI không duplicate rule logic?

- RQ-W2-3: Win detection có thể giới hạn quanh lastMove thay vì scan toàn board không?

- RQ-W2-4: Fixture/test corpus nào đủ ổn định để trở thành correctness baseline nếu implementation language thay đổi ở giai đoạn sau?

2. Background / Literature

TypeScript được dùng ở baseline vì project web ban đầu dùng React/TypeScript và đề cương dự kiến triển khai cả game logic/AI trong cùng ecosystem. [B] TypeScript bổ sung static type checking cho JavaScript, giúp mô tả contract của Player, Move, GameState, RuleSet và result types trước runtime. [B] Vitest cung cấp test runner phù hợp Vite ecosystem và được dùng để lưu correctness evidence.

Ở giai đoạn này mục tiêu không phải chứng minh performance. TypeScript Game Engine đóng vai trò implementation thật theo kế hoạch ban đầu và đồng thời tạo

behavior/fixture baseline có thể dùng làm oracle nếu architecture được đánh giá lại về sau.

3. Methods

3.1. State ownership

TypeScript Game Engine là owner của domain state trong Week 2:

UI/caller → GameEngine.makeMove(command) → validate → transition → terminal check → immutable/read-only snapshot/result.

- Caller không trực tiếp mutate board.
- Rule checking không nằm trong React component.
- Một move chỉ được commit sau khi validation pass.
- Snapshot/read API không cho phép bypass engine invariant.

3.2. Separation of concerns

- Board: storage/index/access.
- GameState: currentPlayer, status, winner, moveCount, lastMove.
- RuleSet: win condition và rule-specific restrictions.
- GameEngine: lifecycle + transaction boundary.
- WinDetector: line traversal; không tự quyết định mọi rule variant.
- Tests/fixtures: behavioral specification có thể tái sử dụng.

4. Implementation

4.1. Data model reference [D nếu chưa đối chiếu repo]

```
type Player = 1 | -1;

type Cell = 0 | Player;


interface Move {
```

```
  row: number;
  col: number;
  player: Player;
}

interface GameState {
  board: readonly Cell[];
  currentPlayer: Player;
  status: 'playing' | 'won' | 'draw';
  winner: Player | null;
  moveCount: number;
  lastMove: Move | null;
}
```

4.2. Board representation

Logical board là N×N. Contiguous one-dimensional representation có thể dùng:

index = row × N + col.

Với board 15×15, có 225 cells. Encoding phải thống nhất với dataset và migration contract sau này. Nếu repository thật dùng encoding khác thì report phải sửa theo source code, không ép source code khớp reference snippet.

4.3. Move transaction

makeMove(row,col):

- validate bounds;
- validate game status = playing;
- validate target cell empty;
- validate RuleSet-specific constraints;
- apply current player;

- update moveCount và lastMove;

- check win từ lastMove;

- nếu chưa win, check draw;

- nếu vẫn playing, switch player;

- return MoveResult + state snapshot.

4.4. Win detection

Chỉ line đi qua lastMove có thể vừa trở thành winning line sau move mới. Scanner kiểm tra bốn axes:

- horizontal: (0,1);

- vertical: (1,0);

- main diagonal: (1,1);

- anti-diagonal: (1,-1).

Với direction (dr,dc):

count = 1 + countSame(+dr,+dc) + countSame(-dr,-dc).

Line traversal phải tách khỏi win-policy để có thể cấu hình Freestyle/exact-five/project Caro rule mà không duplicate traversal code.

4.5. Public API baseline

- createGame(ruleSet)

- getState()

- makeMove(row,col)

- reset()

- getLegalMoves()

- cloneState()/snapshot() nếu search layer cần.

Legal move và candidate move không đồng nghĩa. Week 2 chỉ định nghĩa legal move theo game rules; candidate generation là search-space optimization ở giai đoạn thuật toán sau.

5. Experimental Setup

Week 2 là correctness experiment, không phải performance benchmark.

Input:

- empty board;
- board với occupied cells;
- horizontal/vertical/diagonal near-win fixtures;
- boundary/corner positions;
- rule-specific overline/block fixtures nếu RuleSet hỗ trợ;
- near-full/full board cho draw;
- invalid commands.

Conditions:

- cùng RuleSet cho expected và actual result;
- deterministic input;
- mỗi fixture ghi expected state/result;
- test độc lập UI/rendering.

Test cases tối thiểu:

- out-of-bounds bị reject;
- occupied cell bị reject;
- move sau terminal bị reject;
- turn switching đúng;
- four win directions;
- boundary không out-of-range;
- draw đúng;
- reset đúng;
- state snapshot không cho caller phá invariant;
- RuleSet edge cases đúng specification.

6. Results / Evidence

[A] Chỉ ghi test count/pass rate, commit hash, file/function hoặc screenshot sau khi đối chiếu repository/test output thật.

Evidence cần link:

- TypeScript Game Engine source/commit.

- Vitest report hoặc CI output.

- Fixture definitions.

- Rule Specification version.

- Diagram state ownership.

- Nếu exact completion week không còn được chứng minh, ghi "implementation evidence hiện tại", không bịa date.

7. Discussion

Week 2 tạo correctness baseline bằng TypeScript đúng với proposal. Giá trị của baseline này không nằm ở việc TypeScript "tốt hơn" hay "chậm hơn", mà ở chỗ semantics của game đã được khóa trước khi AI search xuất hiện.

Nếu Week 4 có migration sang Rust, fixture corpus Week 2 trở thành equivalence oracle: cùng board/rules phải cho cùng legal moves, terminal result và state transition. Đây là cách biến thay đổi công nghệ thành một migration có kiểm chứng thay vì port code rồi hy vọng.

8. Limitations

- Chưa có search workload nên Week 2 không đủ dữ liệu để kết luận về runtime language.

- Unit tests chỉ chứng minh các case đã được mô tả, không chứng minh toàn bộ state space.

- Rule profile chưa khóa đầy đủ thì expected result của edge case cũng chưa thể coi là authoritative.

- UI integration chưa được kiểm tra ở Week 2.

- Nếu repository history không còn TypeScript implementation tương ứng, không được tạo lại một baseline giả và gọi nó historical evidence.

9. Deliverables / Definition of Done

- TypeScript Game Engine chạy độc lập React.

- Board/GameState/RuleSet/MoveResult contracts rõ.

- Move transaction và win detection có test.

- Fixture corpus versioned.

- State ownership diagram tồn tại.

- API đủ ổn định để React/PvP sử dụng ở Week 3.

- Baseline behavior đủ để làm migration oracle nếu architecture thay đổi về sau.

10. References

- [TS-1] TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/

- [VITEST-1] Vitest Guide: https://vitest.dev/guide/

- [VITEST-2] Writing Tests: https://vitest.dev/guide/learn/writing-tests

- [R1] Renju International Federation, International Rules of Gomoku: https://gomoku.renju.net/gomokurules/

.

03 - Tuần 3 - Web UI & PvP

TUẦN 3 - REACT WEB UI, PVP VÀ TÍCH HỢP TYPESCRIPT GAME ENGINE

0. Artifact phải đính kèm

- TypeScript Game Engine baseline và test report từ Tuần 2.
- Engine/GameController interface dùng bởi UI.
- UI wireframe hoặc screen map.
- Deployment link nếu đã có.
- Screenshot desktop/mobile.
- Integration test report.
- Commit/tag hoặc implementation evidence hiện tại.

1. Plan / Research Questions

- RQ-W3-1: Làm thế nào kết nối React UI với TypeScript Game Engine mà không duplicate game rules?
- RQ-W3-2: State nào thuộc domain engine và state nào chỉ thuộc presentation layer?
- RQ-W3-3: Coordinate/input flow nào bảo đảm cùng một user action tạo đúng một domain command?
- RQ-W3-4: Những integration fixtures nào cần giữ lại để Week 4 có thể thay implementation phía sau mà UI contract không đổi?

2. Background / Literature

[B] React khuyến nghị quản lý state theo ownership rõ ràng và nâng state lên nơi cần chia sẻ thay vì tạo nhiều nguồn state độc lập. Trong project này, domain state phải do Game Engine sở hữu; React chỉ render snapshot/result và giữ presentation state.

TypeScript vẫn là implementation language của toàn bộ project ở Week 3, đúng với đề cương ban đầu. Chưa có Rust/Wasm trong architecture của tuần này.

3. Methods

3.1. Ownership boundary

React UI → event handler → GameController/Engine interface → TypeScript Game Engine → MoveResult/GameState snapshot → React render.

- React không tự switch currentPlayer.

- React không tự check win/draw.

- React không tự mutate board.

- React không tự định nghĩa legal move.

- Presentation state như selected menu, dialog open/close, animation state có thể nằm trong React.

3.2. Stable application contract

UI chỉ phụ thuộc một interface ổn định:

- getState()

- makeMove(row,col)

- reset()

- getLegalMoves()

- subscribe()/callback nếu architecture cần reactive update.

Mục đích của interface này ở Week 3 là decouple UI khỏi concrete engine module, không phải vì Rust đã được chọn. Nó trở thành migration seam hữu ích khi architecture được đánh giá lại ở Week 4.

4. Implementation

4.1. Scope UI/PvP

- Board 15×15 responsive theo scope.

- Render player markers.

- Highlight lastMove.

- Current-player indicator.

- Player-vs-Player end-to-end.

- New Game / Reset.

- Win/Draw result display.

- Rule/profile information nếu sản phẩm expose.

- Error/invalid-move feedback nếu cần.

4.2. Interaction flow

User click cell

→ React handler xác định row/col

→ GameController.makeMove(row,col)

→ TypeScript Game Engine validate + transition + terminal check

→ MoveResult/GameState snapshot

→ React render snapshot.

4.3. State ownership invariant

UI state không được trở thành authority cho domain state. Nếu React local state và GameEngine state khác nhau, engine snapshot là nguồn dùng để sửa render; không tạo rule branch riêng trong component để "đồng bộ thủ công".

5. Experimental Setup

Week 3 kiểm tra integration/correctness, không benchmark language runtime.

Input:

- deterministic move sequences;

- invalid move sequences;

- terminal-state sequence;

- reset sequence;

- responsive desktop/mobile layouts.

Conditions:

- cùng move sequence phải tạo cùng final state ở engine unit test và UI integration;

- UI không thay domain state khi engine reject move;

- terminal state chặn move tiếp theo;

- reset trả cả engine và rendered UI về initial state;

- responsive layout không làm sai row/col mapping.

6. Results / Evidence

[A] Chỉ ghi pass/fail, screenshot, deployment URL, source file và commit sau khi đối chiếu source thực.

Evidence cần link:

- TypeScript engine commit/tag hoặc implementation evidence hiện tại.

- React/GameController source.

- Integration test output.

- Desktop/mobile screenshots.

- Deployment.

- Data-flow/ownership diagram.

7. Discussion

Week 3 chứng minh product layer có thể hoạt động trên một domain engine độc lập UI. Separation này làm giảm nguy cơ React component chứa business rule và tạo một stable boundary cho thay đổi implementation phía sau.

Điểm quan trọng về chronology: interface abstraction được tạo để giảm coupling. Nó không được trình bày ngược lịch sử như thể project đã biết trước sẽ chuyển Rust.

8. Limitations

- UI integration test không chứng minh search performance.

- Screenshot chỉ chứng minh presentation/integration, không chứng minh domain correctness nếu không có test.

- Responsive correctness cần device/browser evidence nếu report đưa claim cross-device.

- Week 3 chưa có computational workload đủ để kết luận cần migration runtime.

9. Deliverables / Definition of Done

- PvP chạy end-to-end.

- React không chứa duplicate win/legal-move logic.

- Engine/UI ownership diagram tồn tại.

- Integration tests pass hoặc kết quả thực được lưu.

- Screenshot/deployment evidence được link.

- TypeScript Game Engine + fixtures sẵn sàng làm baseline cho Week 4 nếu architecture thay đổi.

10. References

- [REACT-1] Managing State: https://react.dev/learn/managing-state

- [REACT-2] Sharing State Between Components: https://react.dev/learn/sharing-state-between-components

- [REACT-3] Responding to Events: https://react.dev/learn/responding-to-events

- [REACT-4] Rendering Lists: https://react.dev/learn/rendering-lists

- [REACT-5] Conditional Rendering: https://react.dev/learn/conditional-rendering

- [TS-1] TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/

- [VITEST-1] Vitest Guide: https://vitest.dev/guide/

.

04 - Tuần 4 - Rust Heuristic & Pattern

TUẦN 4 - MIGRATION TYPESCRIPT → RUST/WASM, PATTERN, HEURISTIC VÀ BASELINE BOTS

0. Artifact phải đính kèm

- TypeScript Game Engine + fixtures từ Week 2.
- React/PvP integration evidence từ Week 3.
- Architecture Decision Record cho TypeScript → Rust/Wasm.
- Rust crate/workspace commit nếu đã tồn tại.
- Wasm adapter build/integration evidence nếu đã tồn tại.
- Equivalence test report TS↔Rust.
- Pattern/Heuristic fixture corpus.
- Raw benchmark CSV/JSON nếu có runtime comparison.

1. Plan / Research Questions

Week 4 là điểm architecture được đánh giá lại, không phải điểm Rust "đã tồn tại từ trước".

- RQ-W4-1: Khi bắt đầu computational algorithms, việc chuyển authoritative computational/game core từ TypeScript sang Rust/Wasm có giữ nguyên behavior của engine không?
- RQ-W4-2: "Tối ưu" cần đo bằng runtime metric nào thay vì dùng như nhận xét định tính?
- RQ-W4-3: Pattern Detection và Heuristic Evaluation nên formalize thế nào để phục vụ Rule-Based, Minimax và Alpha-Beta?
- RQ-W4-4: Wasm boundary phải đặt ở đâu để search loop không bị chia nhỏ giữa JavaScript và Rust?
- RQ-W4-5: Random/Rule-Based/Heuristic bots cần định nghĩa thế nào để trở thành baseline khoa học?

2. Background / Literature

2.1. Trigger của migration

Đề cương và Week 1–3 dùng TypeScript cho toàn hệ thống. Khi Pattern Detection, Heuristic Evaluation và search preparation bắt đầu, computational core phải thực hiện repeated board traversal, apply/undo, evaluation và về sau recursive game-tree search. Đây là thời điểm hợp lý để đánh giá lại implementation language.

2.2. Rationale kỹ thuật

[B] Rust ownership/borrowing cung cấp memory-safety guarantees without garbage collection. [B] Rust hướng tới zero-cost abstractions. [B] WebAssembly cho phép module compiled chạy cạnh JavaScript trên web. [B] wasm-bindgen hỗ trợ Rust↔JavaScript interoperability.

Các đặc tính này chỉ hỗ trợ hypothesis rằng Rust phù hợp với computational core. Chúng không chứng minh Rust implementation của project nhanh hơn TypeScript.

2.3. Định nghĩa “optimization”

- Algorithmic optimization: giảm số lượng công việc, ví dụ pruning/candidate reduction/cache reuse.

- Runtime optimization: cùng workload nhưng giảm search time/cost per node hoặc tăng nodes per second.

- System optimization: product và experiment dùng cùng core, giảm duplicated implementations.

3. Methods

3.1. Migration method

Step 1: Freeze TypeScript baseline và fixture corpus.

Step 2: Port Board/RuleSet/SearchState semantics cần cho computational core sang Rust.

Step 3: Chạy differential/equivalence tests.

Step 4: Chỉ sau correctness gate mới cut-over web adapter.

Step 5: Implement Pattern Detection, Heuristic Evaluation và baseline bots trong Rust.

Step 6: Expose coarse-grained commands qua Wasm.

Step 7: Thu metrics và benchmark nếu có TypeScript implementation tương đương.

3.2. Ownership sau migration

React/TypeScript UI

→ EnginePort/RustWasmAdapter

→ Rust Core

- Board/SearchState
- Rule-compatible move semantics
- apply/undo
- terminal detection
- Pattern Detection
- Heuristic Evaluation
- Random/Rule-Based/Heuristic bots
- instrumentation

→ SearchResult/GameSnapshot

→ React render.

Experiment runner phải gọi cùng Rust core, không có “benchmark-only algorithm implementation”.

3.3. Wasm boundary rule

Đúng:

- load_position(...)
- make_move(...)
- evaluate_position(...)
- choose_move(config)

Không thiết kế:

- JS gọi Rust từng cell/pattern.
- Rust gọi JavaScript evaluator ở từng leaf.
- UI tự giữ một bản rule/search logic song song.

4. Implementation

4.1. Rust core module mapping [D nếu chưa đối chiếu repo]

src/

- types.rs
- board.rs
- rules.rs
- pattern.rs
- evaluation.rs
- metrics.rs
- search/random.rs
- search/rule_based.rs
- search/minimax.rs
- search/alpha_beta.rs
- search/iterative.rs
- zobrist.rs

4.2. Pattern model

Mỗi contiguous pattern baseline mô tả bởi:

- player;
- direction;
- runLength;

- openEnds;

- RuleSet/context.

Taxonomy baseline:

- Five/terminal line;

- Open Four;

- Closed Four;

- Open Three;

- Closed Three;

- Open Two;

- immediate win/block.

Tên pattern không đủ làm definition. Fixture phải cho board state + expected classification. Broken threats như XX_XX cần detector/window model riêng; không tuyên bố coverage tactical đầy đủ nếu implementation mới chỉ đếm contiguous runs.

4.3. Heuristic model

$E(s,p) = Attack(s,p) - \lambda \times Defense(s,opponent)$

Attack/Defense là weighted sum của pattern counts. Weight là design parameter, không gọi “optimal” nếu chưa có tuning experiment.

Phải version:

- pattern taxonomy;

- weight set;

- λ;

- terminal score;

- tie-break policy.

4.4. Baseline bots

Random:

- chọn legal/candidate move bằng seeded RNG trong experiment.

Rule-Based:

- immediate win;
- immediate block;
- tactical priority;
- deterministic fallback.

Heuristic:

- evaluate one-ply candidate moves;
- chọn max score;
- deterministic tie-break trong controlled experiment.

5. Experimental Setup

5.1. Correctness migration gate

- Same RuleSet/board encoding.
- Same fixed fixtures.
- Same legal moves.
- Same win/draw/terminal result.
- Same apply/undo invariant.
- Same expected pattern classification.
- Same evaluator score nếu TypeScript evaluator tương đương thực sự tồn tại.
- State serialization round-trip không đổi semantics.

5.2. Runtime benchmark nếu có TypeScript baseline hợp lệ

Controlled:

- same board fixtures;

- same workload/evaluator;

- same number of calls hoặc same search config;

- release/production builds;

- same hardware/runtime environment;

- warm-up documented;

- repeated samples.

Metrics:

CostPerNode = SearchTime / NodesVisited

Throughput = NodesVisited / SearchTime

Speedup = T_TS / T_Rust

Nếu không có historical TypeScript implementation tương đương, không được dựng baseline giả. Khi đó chỉ benchmark Rust configurations và ghi limitation.

6. Results / Evidence

[A] Chỉ điền từ source/test/benchmark thật:

- Rust core commit/tag.

- cargo test output.

- Wasm build/integration output.

- TS↔Rust differential test report.

- Pattern/evaluator fixture results.

- raw CSV/JSON.

- environment metadata.

- graph + link raw data/script.

Không dùng câu “Rust tối ưu hơn” nếu chưa có số đo. Claim hợp lệ phải có dạng: “Trên fixture/config X, implementation A đạt metric Y trong environment Z.”

7. Discussion

Migration có hai giá trị cần tách:

- Correctness/architecture: một core độc lập React và dùng chung cho product/experiment.

- Runtime: chỉ được kết luận sau benchmark.

Nếu Rust có cùng nodesVisited nhưng search time thấp hơn, đó là evidence về runtime implementation. Nếu Alpha-Beta làm nodesVisited thấp hơn Minimax, đó là algorithmic effect. Hai nguyên nhân không được gộp.

8. Limitations

- Wasm boundary có overhead.

- Rust/Wasm tăng build/debug/toolchain complexity.

- Workload nhỏ có thể không hưởng lợi.

- Pattern baseline contiguous chưa đại diện toàn bộ tactical threat space.

- Benchmark một machine/browser không đại diện mọi platform.

- Nếu historical TS algorithm implementation không tồn tại, không thể kết luận language speedup cho chính project.

9. Deliverables / Definition of Done

- Architecture Decision Record đúng chronology TypeScript → Rust tại Week 4.

- Rust core build/test được nếu source thật tồn tại.

- TS↔Rust equivalence gate pass trước cut-over.

- Pattern Detector deterministic.

- Heuristic Evaluation versioned/configurable.

- Random/Rule-Based/Heuristic baseline bots có deterministic experiment mode.

- Wasm adapter coarse-grained.

- Evidence links đầy đủ.

- Minimax Week 5 có thể dùng trực tiếp Rust SearchState/Evaluator.

10. References

- [RUST-1] Rust Book Introduction:
https://doc.rust-lang.org/book/ch00-00-introduction.html

- [RUST-2] Understanding Ownership:
https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

- [WASM-1] MDN WebAssembly Concepts:
https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Concepts

- [WASM-2] wasm-bindgen Guide: https://wasm-bindgen.github.io/wasm-bindgen/

- [WASM-3] wasm-bindgen numeric slice transfer:
https://wasm-bindgen.github.io/wasm-bindgen/reference/types/boxed-number-slices.html

- [GOMOKU-1] Allis, van den Herik & Huntjens (1993), Go-Moku Solved by New Search Techniques:
https://cdn.aaai.org/Symposia/Fall/1993/FS-93-02/FS93-02-001.pdf

- [AI-1] UC Berkeley CS188 adversarial search:
https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

.

05 - Tuần 5 - Rust Minimax

TUẦN 5 - RUST MINIMAX, GAME TREE, DEPTH LIMIT VÀ CONTROLLED BENCHMARK

0. Artifact phải đính kèm

- Rust migration/equivalence evidence từ Week 4.
- Pattern Detector và Heuristic Evaluator test report.
- Tactical fixture corpus.
- Rust core commit/tag hoặc implementation evidence hiện tại.
- Wasm integration build nếu product đã cut-over.
- TypeScript Minimax historical implementation chỉ khi repository thực sự chứng minh nó từng tồn tại.
- Benchmark environment metadata.
- Raw benchmark CSV/JSON sau experiment.

1. Plan / Research Questions

- RQ-W5-1: Minimax depth-limited sử dụng cùng Rust SearchState/Evaluator lựa chọn move như thế nào trên board 15×15?
- RQ-W5-2: Search cost tăng thực nghiệm ra sao khi depth tăng trên fixture corpus?
- RQ-W5-3: Apply/undo có giữ state invariant trong recursive search không?
- RQ-W5-4: Nếu historical TypeScript Minimax tương đương tồn tại, Rust/Wasm implementation thay đổi runtime cost như thế nào dưới cùng workload/config?

2. Background / Literature

Minimax mô hình hóa trò chơi zero-sum hai người bằng game tree. MAX chọn child value lớn nhất; MIN chọn child value nhỏ nhất. Terminal node dùng terminal utility; khi depth limit bằng 0 ở state non-terminal, search dùng heuristic E(s,p).

Với branching factor b và search depth m, full-tree Minimax có time complexity theo bậc O(b^m). Trên Caro 15×15, branching factor ban đầu lớn nên depth tăng nhanh làm

search cost bùng nổ. Đây là lý do Week 5 cần baseline Minimax trước khi đánh giá Alpha-Beta ở Week 6.

[B] UC Berkeley CS188 trình bày Minimax recurrence, adversarial search và complexity O(b^m). Công thức chi tiết được khóa trong tab Mathematical Foundations.

3. Methods

3.1. Search model

Input được khóa gồm:

- SearchState/Board.
- root player.
- depthLimit tính theo ply.
- evaluator version/weights.
- move-generation policy.
- deterministic move ordering/tie-break.
- RuleSet.

Output SearchResult gồm:

- bestMove.
- score.
- nodesVisited.
- leafEvaluations.
- achievedDepth.
- generated/candidate count nếu instrumentation hỗ trợ.

3.2. State simulation

Hai lựa chọn thiết kế có thể tồn tại:

- Clone-per-child: đơn giản hơn nhưng có thể tăng copy/allocation.

- Mutable apply/undo: giảm copy nhưng yêu cầu invariant test chặt.

[D] Reference core chọn apply/undo. Đây không được gọi là "nhanh hơn" trước benchmark. Correctness condition bắt buộc là undo(apply(s,a)) = s theo logical state.

3.3. Boundary

Recursive Minimax nằm trọn trong Rust core. TypeScript chỉ gọi top-level choose/search API và nhận SearchResult; không cross Wasm boundary từng node/leaf.

4. Implementation

4.1. Recurrence

- Terminal win của root: +WIN_SCORE trừ ply nếu muốn ưu tiên thắng sớm.

- Terminal loss: -WIN_SCORE cộng ply nếu muốn trì hoãn thua.

- Draw: 0.

- Depth 0 non-terminal: evaluate(state, root).

- MAX node: lấy max(childValue).

- MIN node: lấy min(childValue).

4.2. Module ownership

[D nếu chưa đối chiếu repo]

- state.rs: SearchState/GameState + apply/undo.

- evaluator.rs: shared heuristic.

- movegen.rs: legal/candidate generation.

- metrics.rs: instrumentation.

- search/minimax.rs: Minimax recursion và root selection.

- wasm adapter: coarse-grained search export, không chứa algorithm logic riêng.

4.3. Correctness tests

- terminal win/loss/draw.
- depth = 0 trả heuristic.
- immediate winning move.
- forced block/forced loss trong horizon phù hợp.
- tiny manually-solvable game trees để verify min/max propagation.
- deterministic result trên fixed fixture.
- state unchanged sau search.
- apply/undo invariant qua nhiều nested plies.

5. Experimental Setup

5.1. Minimax depth experiment

Input fixture strata:

- early-game positions.
- mid-game positions.
- tactical positions.
- near-terminal positions.

Independent variable:

- search depth, ví dụ 1..d trong runtime khả thi.

Controlled variables:

- same RuleSet.
- same evaluator/weights.
- same candidate/legal-move policy.
- same move order/tie-break.

- same Rust build/runtime.

- same fixture.

Dependent variables:

- nodesVisited.

- leafEvaluations.

- searchTime.

- candidate/generated move metrics nếu có.

5.2. Runtime benchmark protocol

- release/production build.

- hardware/CPU/RAM/OS ghi rõ.

- browser/Wasm version nếu đo product path; native Rust version nếu đo headless path.

- warm-up protocol ghi rõ.

- nhiều repetitions mỗi fixture/depth.

- lưu raw samples, không chỉ average.

- báo median và spread/p95 nếu sample đủ.

5.3. TypeScript → Rust comparison nếu baseline hợp lệ

Chỉ thực hiện khi repository chứng minh TypeScript Minimax tương đương từng tồn tại.

Fairness contract:

- same board fixtures.

- same depth.

- same evaluator/weights.

- same move generator.

- same move order.

- same terminal scoring.

- same tie-break.

- equivalent instrumentation semantics.

Correctness gate trước timing:

- root score equal.

- bestMove equal nếu tie-break deterministic.

- nodesVisited equal hoặc instrumentation difference được giải thích.

Metrics:

CostPerNode = SearchTime / NodesVisited

Throughput = NodesVisited / SearchTime

Speedup_TS→Rust = T_TS / T_Rust

Nếu không có legitimate TS Minimax baseline, bỏ language-speed comparison thay vì dựng lại lịch sử giả.

6. Results / Evidence

[A] Chỉ điền sau khi chạy trên repository thật.

Evidence bắt buộc:

- source file/function cho Minimax.

- cargo test/CI output.

- commit/tag.

- fixture corpus version.

- raw CSV/JSON.

- environment metadata.

- graph/table phải link về raw data và config/script sinh graph.

Dataset record tối thiểu:

- runId.

- fixtureId/gamePhase.

- depth.

- algorithm = minimax.

- implementation/runtime.

- evaluatorVersion.

- candidatePolicy.

- bestMove/score.

- nodesVisited/leafEvaluations.

- searchTime.

- repetition.

- environmentId.

Không điền số liệu giả vào tab này. Nếu product đã có code nhưng chưa chạy lại benchmark, Results ghi “chưa có benchmark evidence được xác nhận”.

7. Discussion

Minimax Week 5 đóng vai trò algorithmic baseline cho Week 6. Nếu nodesVisited tăng gần exponential theo depth trên fixture corpus, dữ liệu đó hỗ trợ lý do cần pruning/search-space optimization; không cần phóng đại thành kết luận về toàn bộ state space.

Nếu apply/undo và clone được benchmark riêng, chênh lệch đó là implementation strategy effect, không phải Minimax-vs-Alpha-Beta effect.

Nếu Rust/Wasm được so với TypeScript historical baseline, language/runtime interpretation phải dựa trên cùng nodes/search workload. Nếu cả nodesVisited và algorithm configuration khác nhau, không được quy toàn bộ timing difference cho Rust.

8. Limitations

- Depth-limited Minimax chỉ nhìn trong search horizon; heuristic quyết định leaf quality.

- Candidate generation nếu bật sẽ thay search space và có thể làm baseline không còn full legal-move Minimax.

- Timing chịu ảnh hưởng hardware/runtime/JIT/background scheduling nếu chạy browser.

- Một fixture corpus không đại diện mọi game position.

- TypeScript→Rust speedup không thể kết luận nếu không có historical equivalent implementation.

- Reference code trong tài liệu vẫn là [D] cho tới khi đối chiếu repo và compile/test.

9. Deliverables

- Rust Minimax implementation hoặc repository mapping xác nhận implementation hiện tại.

- correctness corpus pass.

- apply/undo invariant pass.

- deterministic tie-break contract.

- SearchResult instrumentation.

- raw depth benchmark CSV/JSON.

- environment metadata.

- nodes-vs-depth và timing table/chart nếu dữ liệu đã có.

- evidence matrix cập nhật: claim → file/function → test → commit/tag → result → figure/table.

- dataset đủ làm baseline cho Alpha-Beta Week 6.

10. References

- [AI-1] UC Berkeley CS188, adversarial search / Minimax / Alpha-Beta: https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

- [RUST-1] Rust Book - Ownership: https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

- [RUST-TEST] Cargo test: https://doc.rust-lang.org/cargo/commands/cargo-test.html

- [BENCH-1] Criterion.rs: https://bheisler.github.io/criterion.rs/book/

- [NIST-1] Measures of Location: https://www.itl.nist.gov/div898/handbook/eda/section3/eda351.htm

.

06 - Tuần 6 - Rust Alpha-Beta

TUẦN 6 - RUST ALPHA-BETA PRUNING VÀ FAIR COMPARISON VỚI MINIMAX

0. Artifact phải đính kèm

- Rust Minimax source/commit/tag hoặc implementation evidence hiện tại.
- Minimax correctness report Week 5.
- Minimax raw benchmark dataset Week 5.
- Fixed board fixture corpus.
- Evaluator/terminal-score configuration.
- Move generator, move ordering và tie-break baseline.
- Metrics schema.
- Alpha-Beta source/commit/tag sau implementation.
- Raw Minimax-vs-Alpha-Beta CSV/JSON sau experiment.

1. Plan / Research Questions

- RQ-W6-1: Alpha-Beta có giữ nguyên Minimax root value trên cùng state/depth/evaluator/search space hay không?
- RQ-W6-2: Alpha-Beta giảm nodesVisited/cutoff workload thực tế bao nhiêu trên fixture corpus?
- RQ-W6-3: Node reduction chuyển thành search-time reduction ở mức nào trong cùng Rust core?
- RQ-W6-4: Hiệu quả pruning thay đổi ra sao theo position structure và child ordering baseline?

2. Background / Literature

Alpha-Beta Pruning là optimization của Minimax sử dụng hai bounds:

- α: lower bound tốt nhất MAX đã bảo đảm trên path hiện tại.
- β: upper bound tốt nhất MIN đã bảo đảm trên path hiện tại.

Cutoff xảy ra khi $\alpha \geq \beta$ vì các child còn lại không thể thay đổi decision value theo bounds hiện có.

[B] Với cùng search tree/order-independent semantics, correct Alpha-Beta giữ nguyên Minimax value của root. Worst-case complexity vẫn cùng bậc O(b^m). Với ideal/perfect move ordering, lý thuyết thường mô tả effective complexity gần O(b^(m/2)); đây là theoretical best-case behavior, không phải runtime guarantee cho project.

[B] Berkeley CS188 cũng lưu ý action/best move có thể khác khi nhiều actions có cùng value hoặc tie-breaking/order khác. Vì experiment của project cần equivalence chặt, tie-break được khóa deterministic khi so bestMove.

3. Methods

3.1. Controlled-comparison principle

Minimax và Alpha-Beta phải dùng cùng:

- board state/root player.
- depth.
- RuleSet.
- evaluator/weights.
- terminal scoring.
- legal/candidate move set.
- move ordering baseline.
- deterministic tie-break.
- Rust build/runtime.

Independent variable của experiment chính:

- pruning disabled = Minimax.
- pruning enabled = Alpha-Beta.

Move Ordering optimization nâng cao không được thay cùng lúc trong experiment chính Week 6; nó là independent experiment riêng.

3.2. Correctness-first method

Trước timing:

- Minimax.score == AlphaBeta.score trên toàn fixture/depth corpus.
- bestMove giống khi tie-break/order khóa deterministic.
- terminal cases giống.
- state unchanged sau search.
- tiny synthetic trees có expected visitation/cutoff được dùng để verify prune logic.

3.3. Metric definition

Primary algorithmic metrics:

- nodesVisited.
- leafEvaluations.
- cutoffCount.
- skippedChildren nếu implementation đếm được.

System metrics:

- searchTime.
- medianSearchTime.
- p95 nếu sample đủ.

Derived:

NodeReduction = 1 - N_alphaBeta / N_minimax

Speedup = T_minimax / T_alphaBeta

Không dùng từ “nodesPruned” nếu implementation không có definition đo được. Cutoff count và skippedChildren thường rõ hơn.

4. Implementation

4.1. Shared modules

[D nếu chưa đối chiếu repo]

- search/minimax.rs: baseline giữ nguyên.
- search/alpha_beta.rs: Alpha-Beta baseline.
- evaluator.rs: shared evaluator.
- movegen.rs: shared move set.
- metrics.rs: shared instrumentation.
- state.rs: shared apply/undo.
- wasm adapter: top-level search call, không chứa duplicated algorithm logic.

4.2. Alpha-Beta recurrence

Khởi tạo root:

$\alpha = -\infty$

$\beta = +\infty$

MAX node:

- value = max(value, child).
- α = max(α, value).
- cutoff khi $\alpha \geq \beta$.

MIN node:

- value = min(value, child).

- $\beta = \min(\beta, value)$.

- cutoff khi $\alpha \geq \beta$.

4.3. SearchResult

Alpha-Beta trả cùng contract với Minimax để experiment không cần hai pipeline khác nhau:

- bestMove.

- score.

- nodesVisited.

- leafEvaluations.

- cutoffs/skippedChildren.

- achievedDepth.

- search timing được thu theo cùng protocol.

4.4. Preparation cho optimization sau Week 6

Transposition Table, Zobrist, advanced Move Ordering và Iterative Deepening có reference implementation riêng nhưng không được bật trong baseline experiment Minimax-vs-Alpha-Beta nếu mục tiêu là cô lập pruning effect.

5. Experimental Setup

5.1. Fixture corpus

Chia strata:

- early-game.

- mid-game.

- tactical.

- near-terminal.

Mỗi fixture lưu:

- fixtureId.
- board state.
- player to move.
- RuleSet.
- expected tactical property nếu có.

5.2. Depth conditions

Chạy cùng depth set 1..d trong runtime khả thi. Với mỗi fixture/depth:

- chạy Minimax repetitions.
- chạy Alpha-Beta repetitions.
- kiểm score/bestMove equivalence trước khi dùng timing.
- lưu nodes/cutoffs/time raw samples.

5.3. Runtime conditions

- same hardware/OS.
- same Rust release build.
- same process/browser mode cho pair comparison.
- warm-up được ghi rõ.
- repetitions n được ghi rõ.
- không dùng một lần chạy làm benchmark.
- raw samples được giữ nguyên.

5.4. Statistical reporting

Tối thiểu:

- n.
- median.
- mean.

- standard deviation hoặc percentile spread.

- p95 nếu đủ sample.

- raw CSV/JSON.

Nếu timing nhiễu nhưng nodesVisited ổn định, nodesVisited là evidence trực tiếp hơn cho algorithmic pruning; runtime là system-level evidence bổ sung.

6. Results / Evidence

[A] Chỉ điền từ experiment thật.

Evidence matrix cần có:

- Alpha-Beta claim → source file/function.

- equivalence test → test/fixture.

- commit/tag.

- raw experiment CSV/JSON.

- environment metadata.

- table/chart → raw data + generation script/config.

Bảng kết quả nên chứa tối thiểu:

- fixtureId.

- depth.

- algorithm.

- score/bestMove.

- nodesVisited.

- leafEvaluations.

- cutoffs/skippedChildren.

- searchTime sample/median.

- NodeReduction.

- Speedup.

Interpretation hợp lệ chỉ sau dữ liệu, ví dụ:

- “Alpha-Beta visited X% fewer nodes than Minimax trên corpus/config này.”

- “Median runtime giảm theo factor S trong environment này.”

Không được viết trước dữ liệu:

- “Alpha-Beta luôn nhanh hơn X lần.”

- “Project đạt best-case O(b^(m/2)).”

- “Rust làm Alpha-Beta tốt hơn” khi independent variable chỉ là pruning trong cùng Rust core.

7. Discussion

Nếu root value equivalence pass và nodesVisited giảm, dữ liệu hỗ trợ conclusion rằng Alpha-Beta loại bỏ phần search không cần thiết mà không thay minimax decision value trên tested corpus.

Mức runtime reduction có thể không tỷ lệ tuyến tính với node reduction vì mỗi node có evaluator/move-generation cost, branch behavior, cache behavior và measurement noise. Vì vậy report phải trình bày cả algorithmic metric lẫn runtime metric.

Position structure và move order ảnh hưởng mạnh đến pruning. Week 6 phải giữ ordering baseline cố định; Week 7 hoặc experiment tiếp theo mới thay Move Ordering để đo riêng effect đó.

Do cả Minimax và Alpha-Beta nằm trong cùng Rust core, JS↔Wasm boundary giống nhau. Điều này tăng internal validity cho comparison vì chênh lệch không đến từ số lần cross-language calls.

8. Limitations

- Kết quả phụ thuộc fixture corpus và depth range.

- Alpha-Beta effectiveness phụ thuộc move ordering.

- Candidate generation nếu bật làm search space khác full legal-move search.

- Timing phụ thuộc hardware/runtime/environment.

- Ideal O(b^(m/2)) không phải guarantee.

- BestMove equality cần deterministic tie-break; root score equivalence là property cơ bản hơn.

- Week 6 baseline không tự chứng minh TT/Zobrist/Iterative Deepening tốt hơn; mỗi optimization cần experiment riêng.

9. Deliverables

- Rust Alpha-Beta implementation hoặc repository mapping xác nhận implementation hiện tại.

- Minimax↔Alpha-Beta equivalence corpus pass.

- instrumentation definition khóa rõ.

- raw same-state/same-depth dataset.

- nodesVisited/cutoff/timing table.

- NodeReduction và Speedup computed từ raw data.

- graph/chart có provenance.

- Discussion ghi cả pruning mạnh và pruning yếu.

- evidence matrix cập nhật.

- baseline sẵn sàng cho Candidate Generation/Move Ordering/TT/Iterative Deepening experiments.

10. References

- [AI-1] UC Berkeley CS188, adversarial search / Minimax / Alpha-Beta: https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

- [AI-2] UC Berkeley CS188 Project 2, Minimax/Alpha-Beta equivalence and tie-breaking behavior: https://inst.eecs.berkeley.edu/~cs188/fa26/projects/proj2/

- [RUST-1] The Rust Programming Language: https://doc.rust-lang.org/book/

- [RUST-TEST] Cargo test: https://doc.rust-lang.org/cargo/commands/cargo-test.html

- [NIST-1] Measures of Location: https://www.itl.nist.gov/div898/handbook/eda/section3/eda351.htm

.

03 - Rust Core Algorithms Implementation

RUST CORE ALGORITHMS - IMPLEMENTATION SPECIFICATION

0. Evidence Status

[D] Implementation Specification. Tab này định nghĩa architecture, algorithm contracts, invariants, metrics và verification gates cho Rust computational core sau migration ở Week 4. Code cụ thể nằm trong tab “Rust Core Algorithms - Reference”. Chỉ sau khi source được đối chiếu repository thật và cargo test/build/benchmark pass mới được nâng thành [A] Source Evidence.

1. Scope và dependency graph

Authoritative computational core sau Week 4 gồm:

- Board/SearchState và rule-compatible state transitions.
- Pattern Detection.
- Heuristic Evaluation.
- Random Bot.
- Rule-Based Bot.
- Heuristic Bot.
- Minimax.
- Alpha-Beta Pruning.
- Candidate Move Generation.
- Move Ordering.
- Iterative Deepening.
- Zobrist Hashing.
- Transposition Table.
- Search Metrics và Experiment Hooks.

Dependency order:

Board + Rules

→ SearchState + apply/undo

→ Pattern Detection

→ Heuristic Evaluation

→ Baseline Bots

→ Minimax

→ Alpha-Beta

→ Candidate Generation / Move Ordering

→ Zobrist + Transposition Table

→ Iterative Deepening + deadline

→ Experiment Runner / Wasm adapter.

Không cho phép search algorithm tạo một Board/Rule implementation riêng. Mọi algorithm phải sử dụng cùng state/rule semantics.

2. Core invariants

2.1. Board invariant

- Mọi cell có đúng một trong ba trạng thái: Empty, Player X, Player O.

- row/col luôn nằm trong [0, N-1].

- flatten index = row × N + col.

- move_count bằng số occupied cells trong state hợp lệ.

2.2. Apply/Undo invariant

Với state s và legal move a:

s' = apply(s,a)

undo(s', token) = s.

Verification phải so logical equality của toàn state, gồm board, current player, status, move count, last move và hash nếu hash incremental được bật.

2.3. Terminal invariant

Nếu state là terminal thì search không sinh child mới. Terminal utility phải dominate non-terminal heuristic range.

2.4. Search immutability invariant

Sau khi choose_move/search trả về, caller-visible game state phải giống state trước search. Search được phép mutate nội bộ bằng apply/undo nhưng không để lại mutation.

2.5. Determinism invariant

Khi board, RuleSet, algorithm config, evaluator weights, candidate policy, move ordering, tie-break và random seed giống nhau thì controlled experiment phải reproducible.

3. Domain representation

3.1. Player và Move

Player được biểu diễn bằng enum thay vì magic number trong public API. Move chứa row/col. Internal encoding có thể dùng numeric representation nếu benchmark chứng minh cần thiết, nhưng serialization contract phải ổn định.

3.2. Board

Khuyến nghị contiguous memory representation để indexing, scan và serialization rõ ràng. Với board khóa 15×15 có thể dùng fixed-size buffer; nếu RuleSet cần board-size linh hoạt có thể dùng Vec/boxed slice.

3.3. RuleSet

RuleSet phải khóa ít nhất:

- board size;
- win length K;
- overline policy;
- exact-five/at-least-five policy;
- blocked-two-ends semantics nếu project dùng;
- draw condition.

Project-specific Caro rule không được trình bày như universal international standard nếu không có nguồn tương ứng.

4. Pattern Detection contract

Input:

- Board/SearchState.
- Perspective player.
- RuleSet nếu pattern semantics phụ thuộc rule.

Output:

- Pattern counts hoặc structured PatternFeatures.

Baseline contiguous taxonomy:

- Five/terminal line.

- Open Four.

- Closed Four.

- Open Three.

- Closed Three.

- Open Two.

Mỗi detection phải tránh double counting bằng một start-of-run/window convention rõ ràng.

Limitation bắt buộc: contiguous-run detector không tự động nhận broken threats như XX_XX hoặc X_XXX. Nếu sản phẩm thực tế có tactical window detector, report phải map source/test riêng; nếu chưa có thì không được tuyên bố full tactical coverage.

5. Heuristic Evaluation contract

Evaluation từ perspective p:

$E(s,p) = Attack(s,p) - \lambda \times Defense(s,opponent)$.

Attack/Defense là weighted sums của pattern features.

Required properties:

- deterministic với cùng config;

- terminal state không dùng heuristic bình thường;

- terminal score nằm ngoài heuristic non-terminal range;

- weight table có version;

- evaluator không mutate state;

- score breakdown có thể được expose cho debug/research nếu cần.

Weight values là design parameters [D] hoặc experiment outputs [A], không phải fact học thuật.

6. Baseline Bots

6.1. Random Bot

Mục tiêu: lower baseline.

- chọn từ legal moves hoặc candidate set được công bố rõ;
- experiment phải dùng fixed seed;
- log seed trong raw output.

6.2. Rule-Based Bot

Priority baseline:

- immediate winning move;
- immediate blocking move;
- tactical priority rules;
- deterministic fallback.

Mọi priority rule phải có fixture test.

6.3. Heuristic Bot

- generate legal/candidate moves;
- apply một move;
- evaluate resulting state;
- undo;

- chọn score tốt nhất theo deterministic tie-break.

Đây là one-ply evaluation baseline, không được gọi là Minimax.

7. Minimax contract

Input:

- SearchState.

- root player.

- depth limit tính theo ply.

- evaluator config.

- move generator.

- deterministic tie-break.

Recurrence:

MAX: V(s) = max V(child)

MIN: V(s) = min V(child)

Depth 0 non-terminal: V(s) ≈ E(s, root).

Required metrics:

- nodesVisited.

- leafEvaluations.

- achievedDepth.

- candidateCount hoặc generatedMoves nếu đo branching.

- searchTime lấy ngoài core hoặc từ instrumentation layer có protocol rõ.

Correctness tests:

- immediate win được chọn;

- forced block fixture;

- terminal utilities đúng sign;

- depth semantics đúng ply;

- state trước/sau search bằng nhau;

- deterministic tie-break.

8. Alpha-Beta contract

Alpha-Beta dùng cùng evaluator, move generator, depth semantics và terminal utility với Minimax.

Cutoff khi $\alpha \geq \beta$.

Correctness property quan trọng:

Trên cùng search space/configuration, Alpha-Beta phải trả cùng minimax root value với Minimax. Best move chỉ bắt buộc giống nếu tie-break/action ordering được khóa.

Metrics bổ sung:

- cutoffs.

- skippedChildren hoặc metric prune được định nghĩa chính xác.

- nodesVisited.

Không dùng "nodesPruned" nếu implementation không materialize subtree và chưa định nghĩa cách đếm.

9. Candidate Move Generation

Candidate Generation là search-space restriction, không phải equivalent pruning.

Input:

- current position;

- radius/pattern/tactical policy.

Output:

$C(s) \subseteq A(s)$.

Metrics:

CandidateRatio = |C(s)| / |A(s)|

CandidateReduction = 1 - |C(s)| / |A(s)|

TacticalRecall = retainedCriticalMoves / totalCriticalMoves.

Một candidate policy giảm branching mạnh nhưng bỏ forced win/block là invalid cho tactical corpus dù runtime thấp hơn.

10. Move Ordering

Move Ordering chỉ thay thứ tự search child, không thay search space.

Baseline priorities có thể gồm:

- TT best move.

- immediate win/block.

- heuristic score.

- killer/history heuristic nếu sau này được thêm và có evidence.

Experiment Alpha-Beta ordering phải giữ các biến khác cố định. Expected effect chủ yếu là nodesVisited/cutoff/searchTime, không phải minimax value.

11. Zobrist Hashing

Mỗi cell/piece có random u64 key cố định theo seed/version.

Full hash:

H(s) = XOR các Z[cell,piece] đang occupied XOR side-to-move key nếu cần.

Incremental apply/undo:

H' = H XOR Z[cell,piece].

Correctness gates:

- full recompute == incremental hash sau mỗi apply;
- undo khôi phục hash cũ;
- cùng board + same side-to-move → cùng hash với cùng key table;
- side-to-move khác phải được phân biệt nếu TT semantics cần;
- collision về lý thuyết vẫn tồn tại, nên report không gọi đây là perfect hash.

12. Transposition Table

TT entry tối thiểu:

- key/hash.
- searched depth.
- value.
- bound type: Exact, Lower, Upper.
- bestMove optional.

Lookup chỉ được reuse entry khi depth/bound semantics phù hợp. Metrics:

- ttLookups.

- ttHits.

- ttStores.

- usableHitRate = usableHits / lookups.

TT correctness phải được test độc lập trước khi dùng để kết luận performance.

13. Iterative Deepening và deadline

Search depth lần lượt 1,2,...,d cho tới budget.

Return value phải đến từ deepest fully completed iteration. Nếu hard time budget là requirement, recursive Alpha-Beta phải có deadline/cancellation check; chỉ check giữa hai depth có thể overshoot budget.

Metrics:

- achievedDepth.

- completedIterations.

- timeout/cancelled flag.

- elapsed time.

14. SearchResult và instrumentation

SearchResult tối thiểu:

- bestMove.

- score.

- nodesVisited.
- leafEvaluations.
- cutoffs/skippedChildren.
- candidateCount/generatedMoves.
- achievedDepth.
- ttLookups/ttHits/ttStores khi bật TT.
- timeout flag khi có time budget.

Performance timing protocol không được trộn Wasm initialization vào search timing nếu research question chỉ đo search core. Nếu đo end-to-end UX latency thì phải ghi rõ đó là metric khác.

15. Wasm/API boundary

Web layer chỉ gọi coarse-grained operations:

- load_position(...)
- make_move(...)
- choose_move(searchConfig)
- evaluate_position(...) nếu UI/debug thật sự cần
- get_snapshot()/result serialization.

Không gọi qua boundary cho từng node/cell/leaf. Recursive search, apply/undo, evaluator, hashing và TT lookup nằm trong Rust.

16. Experiment Runner contract

Experiment Runner phải dùng cùng caro-core crate với product.

Mỗi run lưu:

- runId.
- timestamp nếu cần provenance.
- git commit/tag.
- algorithm.
- optimization flags.
- depth/time budget.
- RuleSet/version.
- evaluator version/weights hash.
- candidate policy/version.
- random seed.
- board fixture ID.
- hardware/OS/runtime/build mode.
- raw metrics/result.

Output ưu tiên CSV/JSON để graph có thể trace ngược về raw data.

17. Verification Gate trước [A]

Bắt buộc trước khi reference implementation được gọi là production evidence:

- cargo fmt --check.
- cargo clippy --all-targets --all-features -- -D warnings.
- cargo test.
- release build pass.
- Wasm build/integration test pass nếu web dùng Wasm.
- apply/undo invariant tests pass.
- rule fixtures pass.

- pattern/evaluator fixtures pass.

- Minimax↔Alpha-Beta root-value equivalence corpus pass.

- candidate tactical-recall corpus pass.

- Zobrist full↔incremental tests pass.

- TT depth/bound tests pass.

- iterative deadline/cancellation tests pass nếu time budget được claim.

- raw benchmark tồn tại trước performance conclusion.

18. Evidence Matrix

Mỗi claim final report phải map theo chuỗi:

Report Claim

→ Rust file/function

→ test/fixture

→ commit/tag

→ raw benchmark/result

→ figure/table trong thesis.

Nếu một mắt xích chưa tồn tại thì claim phải được hạ xuống [C] Inference hoặc [D] Recommendation, không được ghi thành kết quả đã chứng minh.

19. References

- [RUST-1] Rust Book - Introduction: https://doc.rust-lang.org/book/ch00-00-introduction.html

- [RUST-2] Rust Book - Ownership: https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

- [WASM-1] MDN WebAssembly Concepts: https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Concepts

- [WASM-2] wasm-bindgen Guide: https://wasm-bindgen.github.io/wasm-bindgen/

- [AI-1] UC Berkeley CS188 adversarial search: https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

- [AI-2] UC Berkeley CS188 Project 2: https://inst.eecs.berkeley.edu/~cs188/fa26/projects/proj2/

- [H-1] Zobrist (1970), A New Hashing Method With Application for Game Playing: https://research.cs.wisc.edu/techreports/1970/TR88.pdf

Rust Core Algorithms - Reference

RUST CORE ALGORITHMS - REFERENCE IMPLEMENTATION

0. Evidence Status

[D] Đây là reference implementation cho architecture mục tiêu. Code trong tab này chưa được gọi là [A] production evidence cho đến khi được đưa vào repository thật và có cargo test/build/benchmark output. Môi trường dựng tài liệu hiện tại chưa có Rust toolchain nên chưa xác nhận compile tại đây.

Mục tiêu của reference code:

- khóa module boundary và ownership;
- khóa API dùng chung giữa Game Engine, bots và search;
- tránh duplicate Board/Rules/Evaluator giữa Minimax và Alpha-Beta;
- chuẩn bị cho Wasm adapter và experiment runner dùng cùng một Rust core.

1. Workspace mục tiêu

crates/caro-core/src/

- lib.rs
- model.rs
- board.rs
- rules.rs
- state.rs
- pattern.rs
- evaluator.rs
- movegen.rs
- metrics.rs
- zobrist.rs
- transposition.rs

- bot/random.rs
- bot/rule_based.rs
- bot/heuristic.rs
- search/minimax.rs
- search/alpha_beta.rs
- search/ordering.rs
- search/iterative.rs

2. model.rs

```rust
pub const BOARD_SIZE: usize = 15;
pub const CELL_COUNT: usize = BOARD_SIZE * BOARD_SIZE;

#[repr(i8)]
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum Cell {
    Empty = 0,
    X = 1,
    O = -1,
}

impl Cell {
    pub fn opponent(self) -> Self {
        match self {
            Cell::X => Cell::O,
            Cell::O => Cell::X,
```

```
            Cell::Empty => Cell::Empty,
        }
    }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub struct Move {
    pub row: u8,
    pub col: u8,
}

impl Move {
    pub const fn new(row: u8, col: u8) -> Self { Self { row, col } }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum GameStatus {
    Playing,
    Won(Cell),
    Draw,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum WinPolicy {
    AtLeastFive,
    ExactFive,
}
```

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct RuleConfig {
    pub win_policy: WinPolicy,
    pub blocked_two_ends_loses_win: bool,
}
```

3. board.rs

```rust
use crate::model::{Cell, Move, BOARD_SIZE, CELL_COUNT};

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Board {
    cells: [Cell; CELL_COUNT],
}

impl Default for Board {
    fn default() -> Self {
        Self { cells: [Cell::Empty; CELL_COUNT] }
    }
}

impl Board {
    #[inline]
    pub const fn index(row: usize, col: usize) -> usize {
```

```
        row * BOARD_SIZE + col
    }

    #[inline]
    pub fn in_bounds(row: i32, col: i32) -> bool {
        row >= 0 && col >= 0 && row < BOARD_SIZE as i32 && col < BOARD_SIZE as i32
    }

    #[inline]
    pub fn get(&self, row: usize, col: usize) -> Cell {
        self.cells[Self::index(row, col)]
    }

    #[inline]
    pub fn set(&mut self, mv: Move, value: Cell) {
        let i = Self::index(mv.row as usize, mv.col as usize);
        self.cells[i] = value;
    }

    #[inline]
    pub fn is_empty(&self, mv: Move) -> bool {
        self.get(mv.row as usize, mv.col as usize) == Cell::Empty
    }

    pub fn legal_moves(&self) -> impl Iterator<Item = Move> + '_ {
        self.cells.iter().enumerate().filter_map(|(i, c)| {
```

```
            if *c != Cell::Empty { return None; }
            Some(Move::new((i / BOARD_SIZE) as u8, (i % BOARD_SIZE) as u8))
        })
    }
}
```

4. rules.rs

```rust
use crate::{board::Board, model::{Cell, Move, RuleConfig, WinPolicy}};

const AXES: [(i32, i32); 4] = [(0,1), (1,0), (1,1), (1,-1)];

fn count_dir(board: &Board, mv: Move, player: Cell, dr: i32, dc: i32) -> usize {
    let mut r = mv.row as i32 + dr;
    let mut c = mv.col as i32 + dc;
    let mut count = 0;
    while Board::in_bounds(r, c) && board.get(r as usize, c as usize) == player {
        count += 1;
        r += dr;
        c += dc;
    }
    count
}
```

```
fn end_cell(board: &Board, mv: Move, player: Cell, dr: i32, dc: i32) -> Option<Cell>
{
    let n = count_dir(board, mv, player, dr, dc) as i32;
    let r = mv.row as i32 + dr * (n + 1);
    let c = mv.col as i32 + dc * (n + 1);
    if Board::in_bounds(r, c) { Some(board.get(r as usize, c as usize)) } else { None }
}


pub fn is_winning_move(board: &Board, mv: Move, player: Cell, rules: RuleConfig)
-> bool {
    AXES.into_iter().any(|(dr, dc)| {
        let left = count_dir(board, mv, player, -dr, -dc);
        let right = count_dir(board, mv, player, dr, dc);
        let len = 1 + left + right;

        let length_ok = match rules.win_policy {
            WinPolicy::AtLeastFive => len >= 5,
            WinPolicy::ExactFive => len == 5,
        };
        if !length_ok { return false; }

        if rules.blocked_two_ends_loses_win {
            let a = end_cell(board, mv, player, -dr, -dc);
            let b = end_cell(board, mv, player, dr, dc);
            let blocked_a = !matches!(a, Some(Cell::Empty));
            let blocked_b = !matches!(b, Some(Cell::Empty));
            if blocked_a && blocked_b { return false; }
```

```
        }
        true
    })
}
```

5. state.rs

```
use crate::{board::Board, model::{Cell, GameStatus, Move, RuleConfig},
rules::is_winning_move};

#[derive(Debug, Clone)]
pub struct GameState {
    pub board: Board,
    pub current: Cell,
    pub status: GameStatus,
    pub move_count: u16,
    pub last_move: Option<Move>,
    pub rules: RuleConfig,
}

#[derive(Debug, Clone, Copy)]
pub struct UndoToken {
    mv: Move,
    previous_current: Cell,
    previous_status: GameStatus,
    previous_last_move: Option<Move>,
```

```
}

impl GameState {
    pub fn apply(&mut self, mv: Move) -> Result<UndoToken, &'static str> {
        if self.status != GameStatus::Playing {
            return Err("game is terminal");
        }
        if !Board::in_bounds(mv.row as i32, mv.col as i32) {
            return Err("out of bounds");
        }
        if !self.board.is_empty(mv) {
            return Err("occupied");
        }

        let player = self.current;
        let token = UndoToken {
            mv,
            previous_current: self.current,
            previous_status: self.status,
            previous_last_move: self.last_move,
        };

        self.board.set(mv, player);
        self.move_count += 1;
        self.last_move = Some(mv);

        if is_winning_move(&self.board, mv, player, self.rules) {
```

```
            self.status = GameStatus::Won(player);
        } else if self.move_count as usize == crate::model::CELL_COUNT {
            self.status = GameStatus::Draw;
        } else {
            self.current = player.opponent();
        }

        Ok(token)
    }

    pub fn undo(&mut self, token: UndoToken) {
        self.board.set(token.mv, Cell::Empty);
        self.move_count -= 1;
        self.current = token.previous_current;
        self.status = token.previous_status;
        self.last_move = token.previous_last_move;
    }
}
```

Correctness invariant bắt buộc:

- Với mọi legal move a: undo(apply(s,a)) phải khôi phục logical state s.

- Test phải so board, current, status, move_count và last_move; khi incremental Zobrist được tích hợp phải so cả hash.

- UndoToken lưu previous_current trực tiếp, không suy ngược từ terminal/non-terminal state.

6. pattern.rs

```rust
use crate::{board::Board, model::{Cell, BOARD_SIZE}};

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PatternKind {
    Five,
    OpenFour,
    ClosedFour,
    OpenThree,
    ClosedThree,
    OpenTwo,
}

#[derive(Debug, Default, Clone, Copy)]
pub struct PatternCounts {
    pub five: u16,
    pub open_four: u16,
    pub closed_four: u16,
    pub open_three: u16,
    pub closed_three: u16,
    pub open_two: u16,
}

impl PatternCounts {
```

```
    pub fn add(&mut self, kind: PatternKind) {
        match kind {
            PatternKind::Five => self.five += 1,
            PatternKind::OpenFour => self.open_four += 1,
            PatternKind::ClosedFour => self.closed_four += 1,
            PatternKind::OpenThree => self.open_three += 1,
            PatternKind::ClosedThree => self.closed_three += 1,
            PatternKind::OpenTwo => self.open_two += 1,
        }
    }
}

fn classify_run(len: usize, open_ends: u8) -> Option<PatternKind> {
    match (len, open_ends) {
        (5.., _) => Some(PatternKind::Five),
        (4, 2) => Some(PatternKind::OpenFour),
        (4, 1) => Some(PatternKind::ClosedFour),
        (3, 2) => Some(PatternKind::OpenThree),
        (3, 1) => Some(PatternKind::ClosedThree),
        (2, 2) => Some(PatternKind::OpenTwo),
        _ => None,
    }
}

pub fn count_contiguous_patterns(board: &Board, player: Cell) -> PatternCounts {
    let mut out = PatternCounts::default();
    let dirs = [(0_i32,1_i32), (1,0), (1,1), (1,-1)];
```

```
   for row in 0..BOARD_SIZE {
      for col in 0..BOARD_SIZE {
         if board.get(row, col) != player { continue; }
         for (dr, dc) in dirs {
            let pr = row as i32 - dr;
            let pc = col as i32 - dc;
            if Board::in_bounds(pr, pc) && board.get(pr as usize, pc as usize) == player
{ continue; }


            let mut len = 0usize;
            let mut r = row as i32;
            let mut c = col as i32;
            while Board::in_bounds(r, c) && board.get(r as usize, c as usize) == player
{
               len += 1; r += dr; c += dc;
            }


            let before_open = Board::in_bounds(pr, pc) && board.get(pr as usize, pc as
usize) == Cell::Empty;
            let after_open = Board::in_bounds(r, c) && board.get(r as usize, c as usize)
== Cell::Empty;
            let open_ends = before_open as u8 + after_open as u8;
            if let Some(kind) = classify_run(len, open_ends) { out.add(kind); }
         }
      }
   }
   out
```

}

```

Limitation bắt buộc: implementation trên chỉ nhận contiguous runs. Broken threats như XX_XX, X_XXX hoặc window-based tactical motifs cần detector thứ hai trước khi tuyên bố pattern coverage đầy đủ.

7. evaluator.rs

```rust
use crate::{model::Cell, pattern::{count_contiguous_patterns, PatternCounts}, state::GameState};

pub const WIN_SCORE: i32 = 1_000_000_000;

#[derive(Debug, Clone, Copy)]
pub struct EvalWeights {
    pub five: i32,
    pub open_four: i32,
    pub closed_four: i32,
    pub open_three: i32,
    pub closed_three: i32,
    pub open_two: i32,
}

impl Default for EvalWeights {
    fn default() -> Self {
        Self { five: 10_000_000, open_four: 1_000_000, closed_four: 100_000,
```
```

```
            open_three: 10_000, closed_three: 1_000, open_two: 100 }
    }
}

fn weighted(c: PatternCounts, w: EvalWeights) -> i32 {
    c.five as i32 * w.five
        + c.open_four as i32 * w.open_four
        + c.closed_four as i32 * w.closed_four
        + c.open_three as i32 * w.open_three
        + c.closed_three as i32 * w.closed_three
        + c.open_two as i32 * w.open_two
}

pub fn evaluate(state: &GameState, root: Cell, w: EvalWeights) -> i32 {
    match state.status {
        crate::model::GameStatus::Won(p) if p == root => return WIN_SCORE,
        crate::model::GameStatus::Won(_) => return -WIN_SCORE,
        crate::model::GameStatus::Draw => return 0,
        crate::model::GameStatus::Playing => {}
    }
    let own = weighted(count_contiguous_patterns(&state.board, root), w);
    let opp = weighted(count_contiguous_patterns(&state.board, root.opponent()), w);
    own - opp
}
```

Weights là parameter versioned, không phải fact học thuật hay “optimal values”.

8. movegen.rs

```rust
use std::collections::BTreeSet;
use crate::{model::{Cell, Move, BOARD_SIZE}, state::GameState};

pub fn legal_moves(state: &GameState) -> Vec<Move> {
    state.board.legal_moves().collect()
}

pub fn nearby_candidates(state: &GameState, radius: i32) -> Vec<Move> {
    if state.move_count == 0 {
        let c = (BOARD_SIZE / 2) as u8;
        return vec![Move::new(c, c)];
    }

    let mut set = BTreeSet::<(u8,u8)>::new();
    for r in 0..BOARD_SIZE {
        for c in 0..BOARD_SIZE {
            if state.board.get(r,c) == Cell::Empty { continue; }
            for dr in -radius..=radius {
                for dc in -radius..=radius {
                    if dr == 0 && dc == 0 { continue; }
                    let nr = r as i32 + dr;
                    let nc = c as i32 + dc;
                    if crate::board::Board::in_bounds(nr,nc) {
```

```
                    let mv = Move::new(nr as u8, nc as u8);
                    if state.board.is_empty(mv) { set.insert((mv.row,mv.col)); }
                }
            }
        }
    }
    set.into_iter().map(|(r,c)| Move::new(r,c)).collect()
}
```

Candidate generation là search-space restriction nên phải benchmark tactical recall riêng; không được coi nó giống Alpha-Beta pruning.

9. metrics.rs

```rust
#[derive(Debug, Default, Clone, Copy)]
pub struct SearchMetrics {
    pub nodes_visited: u64,
    pub leaf_evaluations: u64,
    pub cutoffs: u64,
    pub skipped_children: u64,
    pub tt_lookups: u64,
    pub tt_hits: u64,
    pub tt_stores: u64,
    pub achieved_depth: u8,
```

}

```

10. bot/random.rs

```rust
use crate::{model::Move, state::GameState};

// Deterministic xorshift để experiment có thể seed lại mà không cần dependency ngoài.
#[derive(Clone, Copy)]
pub struct XorShift64(pub u64);
impl XorShift64 {
    pub fn next(&mut self) -> u64 {
        let mut x = self.0;
        x ^= x << 13; x ^= x >> 7; x ^= x << 17;
        self.0 = x; x
    }
}

pub fn choose_random(state: &GameState, rng: &mut XorShift64) -> Option<Move> {
    let moves: Vec<_> = state.board.legal_moves().collect();
    if moves.is_empty() { None } else { Some(moves[(rng.next() as usize) % moves.len()]) }
}
```
```

11. bot/rule_based.rs

```rust
use crate::{model::Move, state::GameState};

pub fn choose_rule_based(state: &mut GameState) -> Option<Move> {
    let me = state.current;
    let moves: Vec<_> = state.board.legal_moves().collect();

    for &mv in &moves {
        let token = state.apply(mv).ok()?;
        let win = matches!(state.status, crate::model::GameStatus::Won(p) if p == me);
        state.undo(token);
        if win { return Some(mv); }
    }

    let opp = me.opponent();
    for &mv in &moves {
        state.board.set(mv, opp);
        let block = crate::rules::is_winning_move(&state.board, mv, opp, state.rules);
        state.board.set(mv, crate::model::Cell::Empty);
        if block { return Some(mv); }
    }

    moves.into_iter().next()
}
```

12. bot/heuristic.rs

```rust
use crate::{evaluator::{evaluate, EvalWeights}, model::Move,
movegen::nearby_candidates, state::GameState};

pub fn choose_heuristic(state: &mut GameState, weights: EvalWeights) ->
Option<Move> {
    let root = state.current;
    let mut best: Option<(i32, Move)> = None;
    for mv in nearby_candidates(state, 2) {
        let token = state.apply(mv).ok()?;
        let score = evaluate(state, root, weights);
        state.undo(token);
        match best {
            None => best = Some((score, mv)),
            Some((s, _)) if score > s => best = Some((score, mv)),
            _ => {}
        }
    }
    best.map(|(_, mv)| mv)
}
```

13. search/minimax.rs

```rust
use crate::{evaluator::{evaluate, EvalWeights, WIN_SCORE}, metrics::SearchMetrics,
        model::{Cell, GameStatus, Move}, movegen::nearby_candidates, state::GameState};


#[derive(Debug, Clone, Copy)]
pub struct SearchResult {
    pub best_move: Option<Move>,
    pub score: i32,
    pub metrics: SearchMetrics,
}


fn minimax_value(state: &mut GameState, depth: u8, root: Cell, weights: EvalWeights,
            metrics: &mut SearchMetrics, ply: i32) -> i32 {
    metrics.nodes_visited += 1;
    match state.status {
        GameStatus::Won(p) => return if p == root { WIN_SCORE - ply } else { -WIN_SCORE + ply },
        GameStatus::Draw => return 0,
        GameStatus::Playing => {}
    }
    if depth == 0 {
        metrics.leaf_evaluations += 1;
        return evaluate(state, root, weights);
    }
```

```
    let maximizing = state.current == root;
    let mut best = if maximizing { i32::MIN } else { i32::MAX };
    let moves = nearby_candidates(state, 2);
    for mv in moves {
        let token = match state.apply(mv) { Ok(t) => t, Err(_) => continue };
        let v = minimax_value(state, depth - 1, root, weights, metrics, ply + 1);
        state.undo(token);
        best = if maximizing { best.max(v) } else { best.min(v) };
    }
    best
}

pub fn search_minimax(state: &mut GameState, depth: u8, weights: EvalWeights) ->
SearchResult {
    let root = state.current;
    let mut metrics = SearchMetrics::default();
    metrics.achieved_depth = depth;
    let mut best_move = None;
    let mut best_score = i32::MIN;

    for mv in nearby_candidates(state, 2) {
        let token = match state.apply(mv) { Ok(t) => t, Err(_) => continue };
        let score = minimax_value(state, depth.saturating_sub(1), root, weights, &mut
metrics, 1);
        state.undo(token);
        if score > best_score {
            best_score = score;
```

```
            best_move = Some(mv);
        }
    }
    SearchResult { best_move, score: best_score, metrics }
}
```

14. search/alpha_beta.rs

```rust
use crate::{evaluator::{evaluate, EvalWeights, WIN_SCORE}, metrics::SearchMetrics,
        model::{Cell, GameStatus, Move}, movegen::nearby_candidates, state::GameState};
use super::minimax::SearchResult;

fn ab_value(state: &mut GameState, depth: u8, root: Cell, weights: EvalWeights,
        metrics: &mut SearchMetrics, mut alpha: i32, mut beta: i32, ply: i32) -> i32 {
    metrics.nodes_visited += 1;
    match state.status {
        GameStatus::Won(p) => return if p == root { WIN_SCORE - ply } else { -WIN_SCORE + ply },
        GameStatus::Draw => return 0,
        GameStatus::Playing => {}
    }
    if depth == 0 {
        metrics.leaf_evaluations += 1;
        return evaluate(state, root, weights);
```

```
    }

    let maximizing = state.current == root;
    let moves = nearby_candidates(state, 2);
    let total = moves.len();

    if maximizing {
        let mut value = i32::MIN;
        for (i, mv) in moves.into_iter().enumerate() {
            let token = match state.apply(mv) { Ok(t) => t, Err(_) => continue };
            let child = ab_value(state, depth - 1, root, weights, metrics, alpha, beta, ply +
1);
            state.undo(token);
            value = value.max(child);
            alpha = alpha.max(value);
            if alpha >= beta {
                metrics.cutoffs += 1;
                metrics.skipped_children += (total - i - 1) as u64;
                break;
            }
        }
        value
    } else {
        let mut value = i32::MAX;
        for (i, mv) in moves.into_iter().enumerate() {
            let token = match state.apply(mv) { Ok(t) => t, Err(_) => continue };
```

```
            let child = ab_value(state, depth - 1, root, weights, metrics, alpha, beta, ply +
1);
            state.undo(token);
            value = value.min(child);
            beta = beta.min(value);
            if alpha >= beta {
                metrics.cutoffs += 1;
                metrics.skipped_children += (total - i - 1) as u64;
                break;
            }
        }
        value
    }
}

pub fn search_alpha_beta(state: &mut GameState, depth: u8, weights: EvalWeights) ->
SearchResult {
    let root = state.current;
    let mut metrics = SearchMetrics::default();
    metrics.achieved_depth = depth;
    let mut best_move = None;
    let mut best_score = i32::MIN;
    let mut alpha = i32::MIN;

    for mv in nearby_candidates(state, 2) {
        let token = match state.apply(mv) { Ok(t) => t, Err(_) => continue };
        let score = ab_value(state, depth.saturating_sub(1), root, weights, &mut metrics,
```

```
                    alpha, i32::MAX, 1);
        state.undo(token);
        if score > best_score {
            best_score = score;
            best_move = Some(mv);
        }
        alpha = alpha.max(best_score);
    }
    SearchResult { best_move, score: best_score, metrics }
}
```

Correctness gate: cùng state/depth/evaluator/move order/tie-break thì Alpha-Beta phải trả cùng root score với Minimax. Best move chỉ bắt buộc giống khi tie-break được khóa deterministic.

15. search/ordering.rs

```rust
use crate::{evaluator::{evaluate, EvalWeights}, model::Move, state::GameState};

pub fn order_moves(state: &mut GameState, moves: &mut [Move], root:
crate::model::Cell,
                   weights: EvalWeights, maximizing: bool) {
    moves.sort_by_key(|&mv| {
        let token = state.apply(mv).ok();
        let score = token.map(|t| {
            let s = evaluate(state, root, weights);
```

```
            state.undo(t);
            s
        }).unwrap_or(i32::MIN);
        if maximizing { -score } else { score }
    });
}
```

Ordering phải benchmark tách khỏi Alpha-Beta baseline vì nó thay hiệu quả pruning, không thay minimax value.

16. zobrist.rs

```
use crate::{
    model::{Cell, Move, BOARD_SIZE, CELL_COUNT},
    state::GameState,
};

pub struct Zobrist {
    keys: [[u64; 2]; CELL_COUNT],
    side_to_move: u64,
}

impl Zobrist {
    pub fn from_seed(mut seed: u64) -> Self {
        fn next(x: &mut u64) -> u64 {
            *x ^= *x << 13;
```

```
        *x ^= *x >> 7;
        *x ^= *x << 17;
        *x
    }

    let mut keys = [[0u64; 2]; CELL_COUNT];
    for key in &mut keys {
        key[0] = next(&mut seed);
        key[1] = next(&mut seed);
    }

    Self {
        keys,
        side_to_move: next(&mut seed),
    }
}

#[inline]
pub fn piece_key(&self, mv: Move, player: Cell) -> u64 {
    let idx = mv.row as usize * BOARD_SIZE + mv.col as usize;
    let piece_idx = match player {
        Cell::X => 0,
        Cell::O => 1,
        Cell::Empty => panic!("empty cell has no Zobrist piece key"),
    };
    self.keys[idx][piece_idx]
}
```

```
pub fn full_hash(&self, state: &GameState) -> u64 {
    let mut hash = 0u64;

    for row in 0..BOARD_SIZE {
        for col in 0..BOARD_SIZE {
            let player = state.board.get(row, col);
            if player == Cell::Empty {
                continue;
            }
            hash ^= self.piece_key(Move::new(row as u8, col as u8), player);
        }
    }

    if state.current == Cell::O {
        hash ^= self.side_to_move;
    }

    hash
}

#[inline]
pub fn toggle_piece(&self, hash: u64, mv: Move, player: Cell) -> u64 {
    hash ^ self.piece_key(mv, player)
}

#[inline]
```

```
    pub fn toggle_side(&self, hash: u64) -> u64 {
        hash ^ self.side_to_move
    }
}
```

Correctness tests bắt buộc:

- full_hash(state) phải bằng incremental hash sau chuỗi apply hợp lệ.
- Sau undo, incremental hash phải trở lại hash trước apply.
- Cùng board nhưng khác side-to-move phải được phân biệt khi side key thuộc TT contract.
- Cùng fixed seed tạo cùng Zobrist table để experiment reproducible.
- Collision vẫn có xác suất khác 0; Zobrist không được mô tả là perfect hash.

17. transposition.rs

```rust
use std::collections::HashMap;
use crate::model::Move;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Bound { Exact, Lower, Upper }

#[derive(Debug, Clone, Copy)]
pub struct TTEntry {
    pub depth: u8,
    pub value: i32,
    pub bound: Bound,
```

```
    pub best_move: Option<Move>,
}

pub struct TranspositionTable {
    map: HashMap<u64, TTEntry>,
}

impl TranspositionTable {
    pub fn new() -> Self { Self { map: HashMap::new() } }
    pub fn get(&self, key: u64) -> Option<TTEntry> { self.map.get(&key).copied() }
    pub fn store(&mut self, key: u64, entry: TTEntry) { self.map.insert(key, entry); }
}
```

TT integration vào Alpha-Beta phải lưu depth và bound semantics đúng. Hash collision là khả năng lý thuyết; production experiment cần ghi rõ strategy ver17.1.
Alpha-Beta + Transposition Table integration

```
use crate::{
    evaluator::{evaluate, EvalWeights, WIN_SCORE},
    metrics::SearchMetrics,
    model::{Cell, GameStatus, Move},
    movegen::nearby_candidates,
    state::GameState,
    transposition::{Bound, TTEntry, TranspositionTable},
    zobrist::Zobrist,
};
```

```
#[allow(clippy::too_many_arguments)]
pub fn alpha_beta_tt_value(
    state: &mut GameState,
    depth: u8,
    root: Cell,
    weights: EvalWeights,
    metrics: &mut SearchMetrics,
    mut alpha: i32,
    mut beta: i32,
    ply: i32,
    zobrist: &Zobrist,
    tt: &mut TranspositionTable,
) -> i32 {
    metrics.nodes_visited += 1;

    match state.status {
        GameStatus::Won(p) => {
            return if p == root { WIN_SCORE - ply } else { -WIN_SCORE + ply };
        }
        GameStatus::Draw => return 0,
        GameStatus::Playing => {}
    }

    if depth == 0 {
        metrics.leaf_evaluations += 1;
        return evaluate(state, root, weights);
```

```
}

let key = zobrist.full_hash(state);
let alpha_original = alpha;
let beta_original = beta;

metrics.tt_lookups += 1;
let cached = tt.get(key);
if let Some(entry) = cached {
    if entry.depth >= depth {
        metrics.tt_hits += 1;
        match entry.bound {
            Bound::Exact => return entry.value,
            Bound::Lower => alpha = alpha.max(entry.value),
            Bound::Upper => beta = beta.min(entry.value),
        }
        if alpha >= beta {
            return entry.value;
        }
    }
}

let maximizing = state.current == root;
let mut moves = nearby_candidates(state, 2);

if let Some(tt_move) = cached.and_then(|entry| entry.best_move) {
    if let Some(pos) = moves.iter().position(|m| *m == tt_move) {
```

```
        moves.swap(0, pos);
    }
}

let mut best_move: Option<Move> = None;
let mut value = if maximizing { i32::MIN } else { i32::MAX };
let total = moves.len();

for (i, mv) in moves.into_iter().enumerate() {
    let token = match state.apply(mv) {
        Ok(t) => t,
        Err(_) => continue,
    };

    let child = alpha_beta_tt_value(
        state,
        depth - 1,
        root,
        weights,
        metrics,
        alpha,
        beta,
        ply + 1,
        zobrist,
        tt,
    );
    state.undo(token);
```

```
    if maximizing {
        if child > value {
            value = child;
            best_move = Some(mv);
        }
        alpha = alpha.max(value);
    } else {
        if child < value {
            value = child;
            best_move = Some(mv);
        }
        beta = beta.min(value);
    }

    if alpha >= beta {
        metrics.cutoffs += 1;
        metrics.skipped_children += (total - i - 1) as u64;
        break;
    }
}

let bound = if value <= alpha_original {
    Bound::Upper
} else if value >= beta_original {
    Bound::Lower
} else {
```

```
        Bound::Exact
    };

    tt.store(
        key,
        TTEntry {
            depth,
            value,
            bound,
            best_move,
        },
    );
    metrics.tt_stores += 1;

    value
}
```

Research note:

- Phiên bản trên dùng full_hash tại mỗi node để ưu tiên correctness/clarity. Production optimization có thể duy trì hash incrementally trong SearchContext sau khi full-hash↔incremental equivalence tests pass.

- TT lookup không được thay đổi minimax value. Nếu bật TT làm root value khác baseline trên cùng corpus thì implementation sai hoặc TT semantics/config không tương đương.

- TT best move có thể được dùng như move-ordering hint mà không đổi search space.

ification/replacement.

18. search/iterative.rs

```
use std::time::{Duration, Instant};
use crate::{
    evaluator::{evaluate, EvalWeights, WIN_SCORE},
    metrics::SearchMetrics,
    model::{Cell, GameStatus, Move},
    movegen::nearby_candidates,
    state::GameState,
};
use super::minimax::SearchResult;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum SearchAbort {
    Deadline,
}

struct SearchDeadline {
    start: Instant,
    budget: Duration,
}

impl SearchDeadline {
    #[inline]
    fn expired(&self) -> bool {
        self.start.elapsed() >= self.budget
    }
```

```
}

#[allow(clippy::too_many_arguments)]
fn ab_deadline(
    state: &mut GameState,
    depth: u8,
    root: Cell,
    weights: EvalWeights,
    metrics: &mut SearchMetrics,
    mut alpha: i32,
    mut beta: i32,
    ply: i32,
    deadline: &SearchDeadline,
) -> Result<i32, SearchAbort> {
    if deadline.expired() {
        return Err(SearchAbort::Deadline);
    }

    metrics.nodes_visited += 1;

    match state.status {
        GameStatus::Won(p) => {
            return Ok(if p == root { WIN_SCORE - ply } else { -WIN_SCORE + ply });
        }
        GameStatus::Draw => return Ok(0),
        GameStatus::Playing => {}
    }
```

```
if depth == 0 {
    metrics.leaf_evaluations += 1;
    return Ok(evaluate(state, root, weights));
}

let maximizing = state.current == root;
let moves = nearby_candidates(state, 2);
let total = moves.len();
let mut value = if maximizing { i32::MIN } else { i32::MAX };

for (i, mv) in moves.into_iter().enumerate() {
    if deadline.expired() {
        return Err(SearchAbort::Deadline);
    }

    let token = match state.apply(mv) {
        Ok(t) => t,
        Err(_) => continue,
    };

    let child_result = ab_deadline(
        state,
        depth - 1,
        root,
        weights,
        metrics,
```

```
            alpha,
            beta,
            ply + 1,
            deadline,
        );

        // Undo luôn phải chạy trước khi propagate timeout/error.
        state.undo(token);
        let child = child_result?;

        if maximizing {
            value = value.max(child);
            alpha = alpha.max(value);
        } else {
            value = value.min(child);
            beta = beta.min(value);
        }

        if alpha >= beta {
            metrics.cutoffs += 1;
            metrics.skipped_children += (total - i - 1) as u64;
            break;
        }
    }

    Ok(value)
}
```

```
fn search_depth_with_deadline(
    state: &mut GameState,
    depth: u8,
    weights: EvalWeights,
    deadline: &SearchDeadline,
) -> Result<SearchResult, SearchAbort> {
    let root = state.current;
    let mut metrics = SearchMetrics::default();
    metrics.achieved_depth = depth;

    let mut best_move: Option<Move> = None;
    let mut best_score = i32::MIN;
    let mut alpha = i32::MIN;

    for mv in nearby_candidates(state, 2) {
        if deadline.expired() {
            return Err(SearchAbort::Deadline);
        }

        let token = match state.apply(mv) {
            Ok(t) => t,
            Err(_) => continue,
        };

        let score_result = ab_deadline(
            state,
```

```
            depth.saturating_sub(1),
            root,
            weights,
            &mut metrics,
            alpha,
            i32::MAX,
            1,
            deadline,
        );

        state.undo(token);
        let score = score_result?;

        if score > best_score {
            best_score = score;
            best_move = Some(mv);
        }
        alpha = alpha.max(best_score);
    }

    Ok(SearchResult {
        best_move,
        score: best_score,
        metrics,
    })
}
```

```
pub fn iterative_deepening(
    state: &mut GameState,
    max_depth: u8,
    budget: Duration,
    weights: EvalWeights,
) -> Option<SearchResult> {
    let deadline = SearchDeadline {
        start: Instant::now(),
        budget,
    };

    let mut deepest_complete: Option<SearchResult> = None;

    for depth in 1..=max_depth {
        if deadline.expired() {
            break;
        }

        match search_depth_with_deadline(state, depth, weights, &deadline) {
            Ok(result) => deepest_complete = Some(result),
            Err(SearchAbort::Deadline) => break,
        }
    }

    deepest_complete
}
```

Correctness/experiment rules:

- Kết quả trả về phải thuộc deepest fully completed depth, không lấy partial root result từ iteration bị timeout.

- Mọi recursive branch phải undo state trước khi propagate SearchAbort.

- Deadline-check frequency là implementation parameter; benchmark overhead và overshoot distribution nếu project claim strict latency.

- Nếu tích hợp TT/move ordering vào iterative deepening, phải version config và benchmark từng optimization layer riêng.

19. lib.rs public API

```rust
pub mod model;
pub mod board;
pub mod rules;
pub mod state;
pub mod pattern;
pub mod evaluator;
pub mod movegen;
pub mod metrics;
pub mod zobrist;
pub mod transposition;
pub mod bot;
pub mod search;

pub use model::*;
pub use state::GameState;
```

20. Compile/Test Gate trước khi nâng thành [A]

Bắt buộc:

- cargo fmt --check
- cargo clippy --all-targets --all-features -- -D warnings
- cargo test
- test apply → undo khôi phục state byte/logical-equivalent;
- test 4 hướng win + edge/corner + exact-five/overline/block-two-ends theo RuleConfig đã khóa;
- pattern fixtures cho open/closed runs và broken-threat detector khi được thêm;
- Minimax vs Alpha-Beta root score equivalence trên fixed corpus;
- deterministic tie-break test;
- candidate tactical recall corpus;
- Zobrist full hash == incremental hash sau apply/undo;
- TT bound/depth tests;
- iterative deepening deadline/cancellation tests;
- raw benchmark data trước mọi performance conclusion.

21. Implementation Order chính thức

Rust Core Engine

→ Rules/State correctness

→ Wasm/EnginePort integration

→ Pattern Detection

→ Heuristic Evaluation

→ Random / Rule-Based / Heuristic baselines

→ Minimax

→ Alpha-Beta

→ Candidate Move Generation

→ Move Ordering

→ Zobrist Hashing

→ Transposition Table

→ Iterative Deepening + deadline

→ Experiment Runner

→ controlled experiments

→ final report evidence.

Lý do thứ tự này: mỗi lớp chỉ phụ thuộc lớp đã có correctness evidence trước nó. Không benchmark search trước khi evaluator/state ổn định, và không benchmark optimization khi baseline chưa khóa.

99 - Reading Index & References

READING INDEX & REFERENCES

Mục đích: đây là danh sách nguồn trung tâm để đọc và trích dẫn. Ưu tiên nguồn primary/official trước tutorial. Không cần đọc toàn bộ mọi tài liệu từ đầu đến cuối; mỗi mục ghi phần nên đọc cho đúng giai đoạn.

A. Research methodology / scientific writing

[RW-1] Purdue OWL - Organization at the Document Level

https://owl.purdue.edu/owl/graduate_writing/introduction_to_writing/documents/drafting-your-document/organization-at-the-document-level.pdf

Dùng cho: hiểu IMRaD và việc điều chỉnh structure theo discipline/genre.

[RW-2] A Guide for Writing in the Scientific Forum

https://pmc.ncbi.nlm.nih.gov/articles/PMC3052777/

Dùng cho: Methods phải đủ chi tiết để người khác tái lập study; áp dụng vào benchmark protocol, fixture, environment và config.

B. Gomoku / game solving / related work

[G-1] Renju International Federation - International Rules of Gomoku

https://gomoku.renju.net/gomokurules/

Đọc: board/rules/win condition. Dùng để phân biệt rule quốc tế với project-specific Caro rule.

[G-2] Allis, van den Herik & Huntjens (1993) - Go-Moku Solved by New Search Techniques

https://cdn.aaai.org/Symposia/Fall/1993/FS-93-02/FS93-02-001.pdf

Đọc: abstract, rules, threat definitions, Threat-Space Search overview, conclusion.

[G-3] Allis (1994) - Searching for Solutions in Games and Artificial Intelligence

https://doi.org/10.26481/dis.19940923la

Đọc: Gomoku/game-solving/search sections khi viết literature review.

[G-4] Allis, van der Meulen & van den Herik (1994) - Proof-Number Search

https://doi.org/10.1016/0004-3702(94)90004-3

Đọc mở rộng; không nằm trong implementation bắt buộc nếu scope vẫn là Minimax/Alpha-Beta.

C. Adversarial search

[AI-1] UC Berkeley CS188 - Adversarial Search / Minimax / Alpha-Beta notes

https://inst.eecs.berkeley.edu/~cs188/fa23/assets/notes/cs188-fa23-note06.pdf

Đọc: Minimax recurrence, O(b^m), Alpha-Beta, ordering effect.

[AI-2] UC Berkeley CS188 Project 2

https://inst.eecs.berkeley.edu/~cs188/fa26/projects/proj2/

Đọc: practical correctness condition rằng Alpha-Beta value phải tương đương Minimax; chú ý tie-breaking/action behavior.

D. TypeScript / React reference phase

[TS-1] TypeScript Handbook

https://www.typescriptlang.org/docs/handbook/

Đọc: types, object types, functions, modules và cách mô tả contract cho GameState/RuleSet/API.

[TS-2] Vitest Guide

https://vitest.dev/guide/

Đọc: test/expect, fixtures và test organization cho correctness baseline Week 2.

[TS-3] Vitest - Writing Tests

https://vitest.dev/guide/learn/writing-tests

Đọc: test structure và assertions để lưu fixture corpus làm migration oracle.

[REACT-1] React - Managing State

https://react.dev/learn/managing-state

[REACT-1] React - Managing State

https://react.dev/learn/managing-state

[REACT-2] React - Sharing State Between Components

https://react.dev/learn/sharing-state-between-components

[REACT-3] React - Responding to Events

https://react.dev/learn/responding-to-events

E. Rust core

[RUST-1] The Rust Programming Language

https://doc.rust-lang.org/book/

Đọc như reference nền tảng.

[RUST-2] Rust Book - Introduction

https://doc.rust-lang.org/book/ch00-00-introduction.html

Đọc: safety/performance motivation, zero-cost abstractions.

[RUST-3] Rust Book - Understanding Ownership

https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

Đọc: ownership, borrowing, memory safety without garbage collector.

[RUST-4] What Is Ownership?

https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html

Đọc kỹ trước apply/undo mutable search state.

[RUST-5] Cargo test

https://doc.rust-lang.org/cargo/commands/cargo-test.html

Đọc: unit/integration tests.

F. WebAssembly / Rust-JavaScript boundary

[WASM-1] MDN WebAssembly

https://developer.mozilla.org/en-US/docs/WebAssembly

Đọc: Wasm role trong web platform.

[WASM-2] MDN WebAssembly Concepts

https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Concepts

Đọc: fast/efficient/portable goal và JS + Wasm architecture.

[WASM-3] wasm-bindgen Guide

https://wasm-bindgen.github.io/wasm-bindgen/

Đọc: export Rust APIs, JS interop, generated TypeScript bindings.

[WASM-4] wasm-bindgen Number Slices

https://wasm-bindgen.github.io/wasm-bindgen/reference/types/number-slices.html

Đọc: numeric slices và typed-array views.

[WASM-5] wasm-bindgen Boxed Number Slices

https://wasm-bindgen.github.io/wasm-bindgen/reference/types/boxed-number-slices.html

Đọc: các trường hợp dữ liệu được copy qua boundary.

[WASM-6] wasm-bindgen-test

https://wasm-bindgen.github.io/wasm-bindgen/wasm-bindgen-test/usage.html

Đọc: browser/Wasm test strategy.

G. Benchmark / statistics

[B-1] Criterion.rs

https://bheisler.github.io/criterion.rs/book/

Đọc: statistics-driven microbenchmark. Không dùng cloud CI timing làm bằng chứng chính nếu môi trường nhiễu.

[B-2] NIST - Measures of Location

https://www.itl.nist.gov/div898/handbook/eda/section3/eda351.htm

Đọc: mean và median.

[B-3] NIST - Confidence Intervals for Proportions

https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm

Đọc: confidence interval cho win-rate/binomial proportion.

H. Hashing / transposition preparation

[H-1] Zobrist, A. L. (1970) - A New Hashing Method With Application for Game Playing

https://research.cs.wisc.edu/techreports/1970/TR88.pdf

Đọc: hash coding for board games và incremental XOR idea.

I. Reading order thực tế

Buổi 1 - Research foundation:

- RW-1.
- G-1.
- G-2 abstract/rules/conclusion.

Buổi 2 - Search theory:

- AI-1 Minimax + Alpha-Beta.
- Mathematical Foundations tab song song.

Buổi 3 - Rust migration:

- RUST-2.
- RUST-3.

- WASM-2.

- WASM-3.

Buổi 4 - Wasm boundary/testing:

- WASM-4.

- WASM-5.

- RUST-5.

- WASM-6.

Buổi 5 - Experiment methodology:

- B-1.

- B-2.

- B-3 khi đến Bot-vs-Bot/win rate.

J. Citation rule cho final thesis

- Dùng primary paper cho historical/research claims khi có thể.

- Dùng official docs cho language/runtime/tool behavior.

- Dùng teaching source như Berkeley để giải thích algorithm, nhưng nếu cần claim lịch sử/original contribution thì tìm primary source.

- Không dùng blog random để chứng minh “Rust nhanh hơn JS” hoặc “Alpha-Beta nhanh hơn X lần”. Các claim hiệu năng của chính project phải dựa trên benchmark project.

- Mọi URL trong weekly log nên được chuyển thành bibliography entry theo format trường yêu cầu khi viết final report.